An Agent-Based Limit Order Book and Reinforcement-Learning Executor for Stealth-Execution Research

Complete Mathematical, Algorithmic, Architectural, and Empirical Reference for the `order_book` Codebase

Manu

October 2026

Abstract

This document is the self-contained technical manual and research record for the study *Hiding in the Order Flow: Learning to Execute Against Liquidity Providers that Detect Metaorders* and its accompanying Python package `order_book`. It is written for two audiences at once: (i) computer scientists, machine-learning researchers, and software engineers who may have *no prior background in market microstructure or quantitative finance*, and (ii) market-microstructure researchers who require exact mathematical equations, algorithmic pseudocode, data-structure invariants, and empirical tables ready to be lifted into a paper's methodology and results sections.

We begin with a pedagogical primer on limit order books, metaorders, implementation shortfall, mechanical market impact, and information leakage (§1). We then map the entire codebase file by file, explaining what data structures each module uses, why those structures were chosen, and how data and control flow through a single simulation step (§2). Next we specify the matching engine (`OrderBook`), accounting and routing layer (`Market`), rolling public state (`PublicState`), and tree-structured random-number architecture that guarantees exact byte-for-byte replay and common-random-number counterfactuals (§§3–6). We derive the exact mathematical update equations and decision rules of every background agent (`FundamentalValue`, `NoiseFlow`, `Speculators`, `InstitutionalTrader`, `MarketMaker`, `OrderAnticipator`) and every metaorder detector (the rule-based A1 `FlowDetector`, the four held-out A2 statistical detectors, and the learned A3 multinomial logistic classifier). We then document the twelve classical, model-based, and stealth-heuristic execution baselines, the A0 calibration protocol, the rank-based and paired-bootstrap statistical methodology, and the full results of all validation and Phase 2 measurement experiments (E1, E2, E3, E7, and E7b). Finally, Part IV documents the Phase 3 reinforcement-learning execution environment (`ExecutionEnv`), the 13-dimensional sign-normalised observation encoder, the 15-way action space, the telescoping implementation-shortfall reward theorem, the Proximal Policy Optimisation (`PPOTrainer`) implementation, the parallel Weights & Biases hyperparameter sweep (`sweep_rl.py`), and its results, which show the learned policy ahead of TWAP on the seeds it was selected on but not on unseen ones. Part V documents the two-world counterfactual machinery that later studies build on: the ghost arm, four independently switchable reaction channels and a counter-based paired-seed protocol.

Contents

I Conceptual Primer and Codebase Navigation **4**

1 A Self-Contained Primer for Non-Finance Readers **4**

1.1 What is a Limit Order Book (LOB)? 4

1.2 What is Optimal Execution and a Metaorder? . . . 4
1.3 How Do We Measure Execution Cost? (Implementation Shortfall) . . . 5
1.4 Mechanical Market Impact vs. Information Leakage (The Core Distinction) . . . 5

2 Codebase Map, Directory Layout, and Data Flow 6
2.1 Directory and Module Inventory . . . 6
2.2 Strict Downward Import Hierarchy . . . 7
2.3 Anatomy of a Single Simulation Step (`Simulator.step()`) . . . 7

II The Simulator: Data Structures, Equations, and Validation 8

3 Matching Engine and Data Structures (`order_book.book`) 8
3.1 Internal Data Structures and Complexity Guarantees . . . 8
3.2 Bounded Price Window and Automatic Eviction . . . 9
3.3 Matching, Self-Trade Prevention (STP), and Order Modification . . . 9

4 Accounting, Reservations, and Routing 10
4.1 Exact Reservation Bookkeeping . . . 10
4.2 The Counterfactual Visibility Switch (`Market.hide(cid)`) . . . 10

5 Rolling Public State (`order_book.public_state`) 11

6 Randomness Architecture and Common Random Numbers (`order_book.simulator`) 12

7 Mathematical Specification of Every Background Agent (`order_book.agents`) 12
7.1 Hidden Fundamental Value (`agents/fundamental.py`) . . . 12
7.2 Zero-Intelligence Noise Flow (`agents/noise.py`) . . . 13
7.3 Fundamentalists, Chartists, and Brock–Hommes . . . 13
7.4 Institutional Traders and Metaorders . . . 14
7.5 Market Makers with Detection and Fragility . . . 15
7.6 Order Anticipator (`agents/anticipator.py`) . . . 16

8 Simulator Validation: Stylised Facts and Square-Root Impact (Phase 1) 16
8.1 Paired-Injection Square-Root Impact Methodology (`order_book.impact`) . . . 16
8.2 Phase 1 Validation Results (`outputs/validation_default.json`) . . . 16

9 Specification of All Detectors 16
9.1 Level A1: Rule-Based Flow Imbalance Detector (`detectors/flow.py`) . . . 17
9.2 Public Feature Tracker (`detectors/features.py`) . . . 17
9.3 Level A2: Four Held-Out Statistical Detectors (`detectors/statistical.py`) . . . 18
9.4 Level A3: Learned Multinomial Logistic Classifier (`detectors/learned.py`) . . . 18

III Phase 2: Execution Baselines, Counterfactual Methodology, and Empirical Results 18

10 Execution Task and Baseline Strategies 19

11 Phase 2 Experimental Results: E2, E3, E7, and E7b 19
11.1 Experiments E2 and E3: The Cover Finding . . . 19
11.2 Experiment E7 (`outputs/stealth_execution/e7/summary.json`): Mapping Leakage Across Market Regimes . . . 21

11.3 Experiment E7b (`outputs/stealth_execution/e7b/summary.json`): Detector-Strength Sweep in `low_cover` 21

IV Phase 3: Reinforcement-Learning Executor, PPO Trainer, and Sweep Results **22**

12 Mathematical Formulation of the Execution MDP **22**
12.1 Observation Space ($\mathcal{S} \subset \mathbb{R}^{13}$, `execution/features.py`) 22
12.2 Discrete Factored Action Space ($\mathcal{A} = \{0, \ldots, 14\}$, `execution/features.py`) 23
12.3 Telescoping Implementation-Shortfall Reward (`execution/env.py`) 24

13 Actor–Critic Network and PPO Training Algorithm (`policy.py`, `ppo.py`) **24**
13.1 Network Architecture (`PolicyNet` and `PolicyStrategy`) 24
13.2 GAE and Clipped PPO 24

14 Phase 3 Hyperparameter Sweep and Empirical Results (`outputs/stealth_execution/rl/sweep/`) **25**

V The Counterfactual Machinery: Reaction Channels, the Ghost Arm, Paired Seeds **25**

15 Reaction Channels and Arms **26**

16 The Ghost Arm **27**

17 Realising the Channels **28**

18 A Paired-Seed Protocol for Agent-Based Counterfactuals **29**

VI Verification, Test Suite, and Complete Reproduction Guide **30**

19 Automated Test Suite (`tests/`) **30**

20 Complete Command Reference for Reproducing Every Phase **31**

Part I
Conceptual Primer and Codebase Navigation

1 A Self-Contained Primer for Non-Finance Readers

If your background is in reinforcement learning, multi-agent systems, or computer science rather than finance, this section defines every domain concept used in the project from first principles.

1.1 What is a Limit Order Book (LOB)?

Modern electronic financial exchanges (such as NASDAQ, NYSE, or Binance) do not have a central dealer who sets a single price. Instead, they run a deterministic data structure called a **Limit Order Book (LOB)** that matches buyers and sellers directly.

- **Ticks and lots.** To avoid floating-point rounding ambiguities, prices and quantities are discrete integers. The smallest price increment is one **tick** (for example, \$0.01). In our simulator, the initial price is 10 000 ticks (which you can think of as \$100.00). The smallest quantity unit is one **lot** (or "share"). Cash is measured in ticks × lots.
- **Sides (`"bid"` vs. `"ask"`).** A **bid** is an order to *buy*. An **ask** (or offer) is an order to *sell*.
- **Limit orders vs. market orders.**
  - A **limit order** specifies a side, a quantity q, and a worst acceptable price p. If a buyer submits a limit bid at $p = 9\,999$ while the cheapest seller is asking 10 001, no trade can occur immediately; the bid *rests* in the book's bid queue at price 9 999, waiting for a seller. Resting limit orders *supply liquidity* (the trader is acting as a **maker**).
  - A **market order** specifies only a side and a quantity q: "buy q lots right now at whatever prices are currently resting on the opposite side of the book." Market orders (and limit orders priced aggressively enough to cross the opposite side immediately) *consume liquidity* (the trader is acting as an **aggressor** or **taker**).
- **Best bid, best ask, mid-price, and bid–ask spread.** At any instant, the highest resting buy price is the **best bid** b_t, and the lowest resting sell price is the **best ask** a_t. Because any bid with $p \geq a_t$ would have matched immediately upon arrival, a resting book is *never crossed*: $b_t < a_t$ always. From b_t and a_t we define two fundamental quantities:

$$\text{Mid-price:} \quad m_t = \frac{b_t + a_t}{2}, \qquad \text{Spread:} \quad s_t = a_t - b_t \geq 1 \text{ tick}. \tag{1}$$

  If you buy 1 lot immediately with a market order, you pay $a_t = m_t + s_t/2$; if you sell 1 lot immediately, you receive $b_t = m_t - s_t/2$. Thus crossing the spread costs $s_t/2$ ticks per lot relative to the mid-price even before moving the price levels.
- **Price–time priority (FIFO).** When an incoming buy order matches against resting asks, which ask gets filled first? First, the lowest-priced ask (*price priority*). Second, among multiple asks resting at that same lowest price, the one that arrived earliest (*time priority*, or First-In-First-Out, FIFO). Consequently, keeping your place near the front of the queue at a price level (**queue position**) is valuable.

1.2 What is Optimal Execution and a Metaorder?

Suppose an institutional investor (a pension fund or mutual fund) decides to buy $Q = 2\,000$ lots of an asset, when typical resting volume at the best ask is only 15 lots and the whole market trades only $\bar{V} \approx 3$–5 lots per second.

- This large parent intention of size Q is called a **metaorder**.
- If the investor dumps all $Q = 2\,000$ lots into the book as a single market order at $t = 0$ (the `immediate` strategy), the order will wipe out every resting ask level from a_0 all the way up the book, paying terrible prices.

- Instead, an **execution algorithm** (the **executor**) splits the parent order Q into many small **child orders** over a time horizon T (say $T = 2\,000$ steps, making a decision every $N = 5$ steps).

1.3 How Do We Measure Execution Cost? (Implementation Shortfall)

Let $m_{\text{arrival}} = m_{t_0}$ be the mid-price at the moment t_0 when the executor starts its metaorder. Over the horizon, the executor obtains a sequence of child fills (p_k, q_k) with $\sum_k q_k = Q$, achieving a quantity-weighted average fill price

$$\bar{p} = \frac{1}{Q}\sum_k p_k q_k = \frac{\text{Total Notional Paid/Received}}{Q}. \tag{2}$$

The standard industry metric of execution cost is **Implementation Shortfall (IS)**, expressed in **basis points (bps)**, where 1 bp $= 10^{-4} = 0.01\%$:

$$\text{IS (bps)} = \text{sign} \cdot \frac{\bar{p} - m_{\text{arrival}}}{m_{\text{arrival}}} \times 10^4, \qquad \text{sign} = \begin{cases} +1 & \text{if buying (}\texttt{"bid"}\text{)}, \\ -1 & \text{if selling (}\texttt{"ask"}\text{)}. \end{cases} \tag{3}$$

A *positive* IS means the executor paid more than m_{arrival} when buying (or received less than m_{arrival} when selling) — i.e. **positive IS is a cost**. For example, at $m_{\text{arrival}} = 10\,000$ ticks, 1 tick of adverse price movement equals exactly 1 bp of implementation shortfall.

1.4 Mechanical Market Impact vs. Information Leakage (The Core Distinction)

Why is IS > 0 when executing a large metaorder? There are two distinct mechanisms:

1. **Mechanical market impact.** Every child buy order physically removes resting sell orders from the book and shifts market makers' inventories (they become short shares, so they shift their quotes upward to mean-revert their inventory). Even if no agent in the market suspects that a large metaorder is underway, consuming Q lots of liquidity mechanically pushes the price up. Across real equity markets, mechanical peak impact follows the famous empirical **square-root law**: $\Delta m \propto \sigma\sqrt{Q/V}$.
2. **Information leakage (detection cost).** Because a metaorder splits Q into dozens of same-direction child orders over time, it leaves a statistical *footprint* in the public order flow (persistent buy imbalance, long runs of buy trades, regular child sizes). Sophisticated liquidity providers (**detecting market makers**) and predatory traders (**order anticipators**) monitor public order flow to infer whether a metaorder is active. When their **detector** fires ($s_t > 0$):
   - Detecting market makers *skew* their quotes upward, *widen* their ask spread, and *cut* their ask quote size to protect themselves from being adversely selected;
   - Order anticipators *front-run* the metaorder by buying ahead of the executor and selling back into the elevated price.

In real markets, you can never observe what a metaorder *would* have cost if detectors had not spotted it. In our deterministic simulator, we can measure this **exact counterfactual**: we run the exact same random seed twice — once where the executor's trades are **visible** to the public order-flow statistics and detectors ($\text{IS}_{\text{visible}}$), and once where the executor's trades still execute and move prices and inventories identically in the book, but are **hidden** from the public flow window ($\text{IS}_{\text{invisible}}$, via `Market.hide(cid)`). This yields the clean additive decomposition:

$$\underbrace{\text{IS}_{\text{visible}}}_{\text{Total Execution Cost}} = \underbrace{\text{IS}_{\text{invisible}}}_{\text{Mechanical Impact Cost}} + \underbrace{\left(\text{IS}_{\text{visible}} - \text{IS}_{\text{invisible}}\right)}_{\text{Information Leakage Cost}}. \tag{4}$$

2 Codebase Map, Directory Layout, and Data Flow

This section explains how the repository is organised, which files implement which abstractions, the strict downward dependency graph, and the exact sequence of operations inside one simulation step.

2.1 Directory and Module Inventory

Table 1: Complete module map of the `order_book` package.

File / Module	Responsibility and Key Classes / Functions
Core Engine & Market Layer (`order_book/`)	
`order_book/book.py`	Zero-dependency price–time FIFO matching engine (`OrderBook`, `_Order`, `Report`, `Trade`, `OrderView`). Bounded window eviction, 4 STP modes, $O(1)$ best-quote/volume/order lookup.
`order_book/market.py`	Accounting and routing (`Account`, `Market`). Running cash/share reservations, budget/window rejection counters, settlement, analysis tape, and counterfactual `hide(cid)`.
`order_book/public_state.py`	Rolling public statistics (`PublicState`): fast/slow EWMA mid, variance, volume; 3 momentum lookback pairs; 20-step ring-buffer flow imbalance; attention multiplier.
`order_book/config.py`	All frozen configuration dataclasses (`BookConfig`, `PublicStateConfig`, `FlowDetectorConfig`, `FundamentalConfig`, `NoiseFlowConfig`, `SpeculatorConfig`, `InstitutionalConfig`, `MarketMakerConfig`, `OrderAnticipatorConfig`, `PopulationConfig`, `EndowmentConfig`, `SimConfig`).
`order_book/simulator.py`	Top-level simulation coordinator (`Simulator`): `SeedSequence` stream tree, population builder, step loop, `snapshot()`/`restore()`, `reseed()`, `add_external()`, `check_conservation()`.
`order_book/presets.py`	Composable market builders: `adversary(level, threshold, base, detecting)`, `regime(liquidity, institutions, makers, mm_size_scale)`, `private_valuations()`, `no_institutions()`.
`order_book/history.py`	Per-step time-series recorder for analysis plots and animations.
`order_book/validation.py`	Stylised-facts suite and pass/fail table against plan §4.2 targets.
`order_book/impact.py`	Natural-metaorder and paired-injection square-root impact measurement.
`order_book/scoring.py`	Mann–Whitney `roc_auc` with tie handling, `detection_delay`, `false_alarm_rate`, `activity_labels`, `bootstrap_ci`, `paired_difference`, `cvar`, `holm`.
`order_book/experiments.py`	The experiment harness papers build on (studies now live in `papers/`): `AdversarySpec`, `EpisodeSpec`, `EpisodeResult`, `run_episode`, parallel `run_specs`, `E2Config`, `E3Config`, `RegimeSpec`, `E7Config`, `StrengthSpec`, `E7StrengthConfig`.
`order_book/env.py`	Legacy generic-trader Gymnasium wrapper (`OrderBookEnv`) using snapshot banks; superseded for optimal execution by `order_book/execution/env.py`.
Background Agent Population (`order_book/agents/`)	
`agents/base.py`	Abstract base class `Agent(cid, market, rng, cash, shares)` opening an `Account`.
`agents/fundamental.py`	`FundamentalValue`: exogenous Gaussian random-walk value with optional mean reversion and private-valuations switch.
`agents/noise.py`	`NoiseFlow`: pooled zero-intelligence Poisson order flow with binomial cancellation, attention scaling, and wealth-balanced side probability.
`agents/speculators.py`	`Speculators`: vectorised array-backed population of $n = 40$ fundamentalists and chartists with Brock–Hommes logit strategy switching.
`agents/institutional.py`	`InstitutionalTrader`, `MetaorderRecord`, `MetaorderLog`: background metaorder arrivals, 5 execution styles, inventory-reverting side choice, ground-truth labels.
`agents/market_maker.py`	`MarketMaker`: Avellaneda–Stoikov two-sided post-only quoter with queue preservation, volatility fragility/pull cooldown, and linear detector response.

File / Module	Responsibility and Key Classes / Functions
`agents/anticipator.py`	`OrderAnticipator`: predatory/contrarian trader acting on detector signals.
Metaorder Detectors (`order_book/detectors/`)	
`detectors/base.py`	Abstract `Detector` interface (`update(public)` → `signal`, `p_buy`, `p_sell`).
`detectors/flow.py`	`FlowDetector` (A1): EWMA of signed flow imbalance with dead-zone threshold.
`detectors/features.py`	`FeatureTracker`: 10 rolling public features from anonymous trades and book state.
`detectors/statistical.py`	Held-out A2 detectors: `RunsDetector`, `CusumDetector`, `VolumeClockDetector`, `RegularityDetector`.
`detectors/learned.py`	`LearnedDetector` & `LearnedDetectorParams` (A3): 3-class multinomial logistic regression trained by Adam with inverse-frequency class weights.
Execution Task, Baselines, and Phase 3 RL (`order_book/execution/`)	
`execution/task.py`	`ExecutionTask`, `Decision`, `ExecState`, `Strategy`, `ExecutionResult`, `ExecutorAgent`, and `expected_volume_from_tape`.
`execution/strategies.py`	All 12 baseline strategies (`Immediate`, `TwapMarket`, `TwapPassive`, `TwapRandom`, `Pov`, `AlmgrenChriss`, `ObizhaevaWang`, `LimitMarket`, `Footprint`, `Mimic`) + `make_strategy` (supporting `"rl@<path>"`).
`execution/calibration.py`	`FillProber`, `fit_resilience`, `calibrate()`: fits $\sigma, \eta, \gamma_{\text{perm}}, \rho, \text{depth}, p_d$ under A0.
`execution/features.py`	Phase 3 MDP state/action encoding: 13-feature sign-normalised `features(s)`, 15-action grid `ACTIONS`, `decision(a, s)`, `Transition`.
`execution/policy.py`	PyTorch actor–critic `PolicyNet` ($13 \to 64 \to 64$ tanh MLP) and `PolicyStrategy` adapter.
`execution/env.py`	`ExecutionEnv`, `EpisodeInfo`, `phase3_config()`: episode rollout with telescoping IS reward.
`execution/ppo.py`	`PPOConfig`, `PPOTrainer`, `_gae`: clipped PPO with GAE(γ, λ) over full episodes.

2.2 Strict Downward Import Hierarchy

A core architectural invariant of the codebase is that dependencies point strictly downward:

$$\texttt{book} \longleftarrow \texttt{market}, \texttt{public_state} \longleftarrow \texttt{detectors} \longleftarrow \texttt{agents} \longleftarrow \texttt{simulator}$$
$$\longleftarrow \texttt{presets}, \texttt{execution}, \texttt{experiments} \longleftarrow \texttt{scripts}.$$

- `OrderBook` (`book.py`) imports *nothing* from `order_book`; it knows only integer client IDs, order IDs, prices, and quantities.
- `Detector` subclasses (`detectors/`) import only `PublicState`; they have no reference to `OrderBook`, `Market`, or any agent, making it structurally impossible for a detector to cheat by inspecting order IDs or client accounts.
- Background agents and `ExecutorAgent` hold a reference to `Market`, never mutating `OrderBook` directly, so cash and share reservations in `Market` can never go out of sync with resting orders in `OrderBook`.

2.3 Anatomy of a Single Simulation Step (`Simulator.step()`)

Every call to `Simulator.step()` in `order_book/simulator.py` executes the following six-stage pipeline in exact deterministic order:

```
def step(self) -> None:
    self.fundamental.advance()                    # 1. Advance hidden fundamental value v_t
    for agent in self.externals:                  # 2. External agents (ExecutorAgent) act
    first
        agent.act()
    agents = self.agents
    for i in self.rng.permutation(len(agents)):   # 3. Background agents act in random order
        agents[i].act()
```

```
    self.market.end_step()                          # 4. Refresh PublicState & clear step
    accumulators
    public = self.market.public
    for detector in self.detectors.values():        # 5. Update every registered detector from
    PublicState
        detector.update(public)
```

Notice two crucial subtleties of this loop:

1. **Why `externals` act outside `self.rng.permutation(len(agents))`.** If adding an external agent (such as `ExecutorAgent` or the paired-impact injected trader) changed the length of `self.agents`, then `self.rng.permutation` would consume a different number of random bytes on step 1, immediately scrambling the action order of all background agents relative to a no-executor run. Keeping `externals` in a separate list that acts before the permutation preserves common random numbers across paired counterfactual runs (§6).
2. **When `PublicState.step_volume` and `detectors` update.** During stages 2 and 3, every trade that settles in `Market._settle()` accumulates into `Market._buy_volume`, `_sell_volume`, `_flow_buy`, `_flow_sell`, and `_step_trades`. At stage 4, `Market.end_step()` pushes those step totals into `PublicState.update(...)`, increments `Market.step` from t to $t+1$, and zeroes the accumulators. Immediately after, at stage 5, all detectors read the newly updated `PublicState`. Thus, on step $t+1$, when agents act in stages 2 and 3, they observe the `PublicState` and detector signals resulting from step t.

Part II
The Simulator: Data Structures, Equations, and Validation

3 Matching Engine and Data Structures (`order_book.book`)

`OrderBook` (`order_book/book.py`) is a single-instrument, price–time (FIFO) limit order book. Because a single simulation episode executes tens of thousands of order placements, cancellations, and queries, every hot-path operation is engineered to avoid linear scans over resting orders.

3.1 Internal Data Structures and Complexity Guarantees

`OrderBook` maintains five synchronised internal containers:

1. **Per-price FIFO queues:** `_queues: dict[Side, dict[int, OrderedDict[int, _Order]]]`. For each side (`"bid"` or `"ask"`) and each active price p, `_queues[side][p]` is a Python `OrderedDict` mapping `order_id` $\mapsto$ `_Order` (a lightweight `__slots__` object holding `order_id, client_id, side, price, quantity`). Because `OrderedDict` is a doubly-linked hash table in CPython:
   - Appending a new resting order at the back of the FIFO queue is $O(1)$;
   - Inspecting the oldest resting order at the front of the queue (`next(iter(queue.values()))`) is $O(1)$;
   - Deleting *any* order by `order_id` from the middle of the queue upon cancellation is also $O(1)$!
2. **Incremental per-level volume cache:** `_volumes: dict[Side, dict[int, int]]`. Stores the sum of resting quantities at price p, updated incrementally in `_add`, `_remove`, and partial fills in `_match`. Querying `get_volume_at_price(side, p)` is $O(1)$ without summing over orders.
3. **Sorted active price lists:** `_prices: dict[Side, list[int]]`. Maintained in ascending sorted order via `bisect.insort` and `bisect_left` when a price level is created or

emptied. Because the bounded window allows at most $N_{\text{levels}} = 20$ active prices per side, `_prices[side]` has length ≤ 20. The best bid is `_prices["bid"][-1]` ($O(1)$) and the best ask is `_prices["ask"][0]` ($O(1)$).

4. **Global order index:** `_orders: dict[int, _Order]` maps every live `order_id` $\mapsto$ `_Order` in $O(1)$.
5. **Per-client order index:** `_client_orders: dict[int, dict[int, None]]` maps each `client_id` to an insertion-ordered set of its currently resting `order_id`s, making `get_client_orders(cid)` and `cancel_all(cid)` $O(k_{\text{client}})$ in the client's own resting orders rather than $O(N_{\text{book}})$.

3.2 Bounded Price Window and Automatic Eviction

To keep observation arrays fixed-dimensional and prevent abandoned far-away orders from accumulating forever, each side only admits limit prices within $N_{\text{levels}} = 20$ ticks (`num_levels`) of its **anchor** $\mathcal{A}$(side):

$$\mathcal{A}(\text{bid}) = \begin{cases} b_t & \text{if bid side non-empty,} \\ a_t - 1 & \text{if bid empty, ask non-empty,} \\ p_{\text{last}} \text{ (or } m_0 - d_0) & \text{if both sides empty,} \end{cases} \tag{5}$$

$$\mathcal{A}(\text{ask}) = \begin{cases} a_t & \text{if ask side non-empty,} \\ b_t + 1 & \text{if ask empty, bid non-empty,} \\ p_{\text{last}} \text{ (or } m_0 + d_0) & \text{if both sides empty.} \end{cases} \tag{6}$$

The active window for bids is $[\max(1, \mathcal{A}(\text{bid}) - N_{\text{levels}} + 1),\ +\infty)$ and for asks is $[1,\ \mathcal{A}(\text{ask}) + N_{\text{levels}} - 1]$. Whenever an aggressive fill or a price-improving limit order shifts $\mathcal{A}$(side), `OrderBook._refresh(side)` pops all resting orders that now lie outside the new window edge and returns them in `report["evicted_orders"]` so `Market` can immediately release their cash/share reservations.

3.3 Matching, Self-Trade Prevention (STP), and Order Modification

When an order $(c, \text{side}, q, p_{\text{limit}})$ enters `OrderBook._match`:

- While the opposite side has resting prices p_{rest} that cross p_{limit} ($p_{\text{rest}} \leq p_{\text{limit}}$ for a buy, $p_{\text{rest}} \geq p_{\text{limit}}$ for a sell, or unconditionally for a market order where $p_{\text{limit}} = \text{None}$) and $q > 0$:
- Inspect the oldest resting order o_{rest} at p_{rest}.
- **Self-Trade Prevention (STP):** If o_{rest}.client_id $== c$ and mode $\neq$ `"none"`:
  - `"cancel_newest"` (default): cancel the incoming order's remaining quantity $q \leftarrow 0$ and stop matching;
  - `"cancel_oldest"`: remove o_{rest} from the book (recording it in `stp_cancelled_orders`) and continue matching q;
  - `"cancel_both"`: remove o_{rest} *and* cancel the incoming remainder $q \leftarrow 0$.
- Otherwise, execute a trade of size $\Delta q = \min(q, o_{\text{rest}}.\text{quantity})$ at the **resting order's price** p_{rest}. Decrement both quantities; if o_{rest} is exhausted, call `_remove(`o_{rest}`)`.
- After the loop, if the order is a limit order and $q > 0$ remains, insert the remainder into the resting book via `_add`. If it is a market order, any unfilled remainder is cancelled (immediate-or-cancel semantics).
- **Order modification (`modify_order`):** If a client modifies an existing resting order by *reducing* its quantity at the *same* price, its `quantity` is decremented in place and it retains its FIFO queue position. If the quantity is increased or the price is changed, the old order is atomically removed and replaced with a new `order_id` at the back of the target queue.

4 Accounting, Reservations, and Routing (`order_book.market`)

`Market` (`order_book/market.py`) sits between all agents and `OrderBook`. Every agent has a unique integer client ID cid and an `Account` with four integer fields:

$$(\text{cash}_i,\ \text{shares}_i,\ \text{reserved_cash}_i,\ \text{reserved_shares}_i).$$

No short-selling and no borrowing are permitted.

4.1 Exact Reservation Bookkeeping

To prevent an agent from placing ten resting limit bids that collectively exceed its cash if all ten were filled, `Market` reserves funds when an order rests and releases them when it leaves:

- **Placement cap:** When client i submits `Market.limit(cid, side, p, q)`, its quantity is first capped by what its *unreserved* balance can fund:

$$q \leftarrow \min(q,\ \text{cap}), \qquad \text{cap} = \begin{cases} \lfloor (\text{cash}_i - \text{reserved_cash}_i)/p \rfloor & \text{if side} = \texttt{"bid"}, \\ \text{shares}_i - \text{reserved_shares}_i & \text{if side} = \texttt{"ask"}. \end{cases} \tag{7}$$

  For a market buy (`Market.market(cid, "bid", q)`), `Market` walks the resting ask levels (p_ℓ, v_ℓ) in ascending price order and computes the exact affordable quantity before submitting to the book, guaranteeing that settlement can never drive cash negative.
- **Settlement (`Market._settle`):** For every trade $(p, \Delta q, \text{buyer } b, \text{seller } s)$ in `report["trades"]`:

$$\text{cash}_b \mathrel{-}= p\,\Delta q, \qquad \text{shares}_b \mathrel{+}= \Delta q, \tag{8}$$

$$\text{cash}_s \mathrel{+}= p\,\Delta q, \qquad \text{shares}_s \mathrel{-}= \Delta q. \tag{9}$$

  If the incoming aggressor was a bid, the passive maker was a resting ask, so $\text{reserved_shares}_s \mathrel{-}= \Delta q$; if the incoming aggressor was an ask, the passive maker was a resting bid at price p, so $\text{reserved_cash}_b \mathrel{-}= p\,\Delta q$. Then, if $q_{\text{rest}} =$ `report["resting_quantity"]` > 0 rests on the book, the submitter's reservation is incremented ($\text{reserved_cash}_c \mathrel{+}= p\,q_{\text{rest}}$ for a bid, or $\text{reserved_shares}_c \mathrel{+}= q_{\text{rest}}$ for an ask). Finally, every order in `report["evicted_orders"]` and `report["stp_cancelled_orders"]` has its reservation released via `_release`.

This guarantees three global invariants checked by `Simulator.check_conservation()`:

$$\sum_i \text{cash}_i = \text{const}, \qquad \sum_i \text{shares}_i = \text{const}, \tag{10}$$

$$\forall i: \quad \text{reserved_cash}_i = \sum_{o \in \text{bids}(i)} p_o q_o, \qquad \text{reserved_shares}_i = \sum_{o \in \text{asks}(i)} q_o. \tag{11}$$

4.2 The Counterfactual Visibility Switch (`Market.hide(cid)`)

In `Market._settle`, each fill updates two sets of step accumulators:

1. **Total traded volume:** `_buy_volume` and `_sell_volume` always include every fill, regardless of who traded.
2. **Public flow accumulators:** `_flow_buy`, `_flow_sell`, and the anonymous step trade list `_step_trades` include a fill *if and only if* `report["client_id"]` $\notin$ `self.hidden_aggressors`.

Calling `sim.market.hide(executor.cid)` adds the executor's client ID to `hidden_aggressors`. When hidden, the executor's orders still enter the book, match against makers, transfer cash and shares, move the mid-price, and count toward total traded volume, *but* detectors and the public signed-flow ring buffer do not see the executor's aggressive trades. Comparing a run with

`visibility="visible"` against `visibility="invisible"` on the same seed isolates the exact cost of detection (§11.1). The complete flow stays available as `public.imbalance_all`, which is what the executor reads, so hiding it changes what detectors see and not what it decides on. It does not isolate the mechanical cost: a hidden executor still moves the mid, and every reaction to that remains. Part V separates them.

5 Rolling Public State (`order_book.public_state`)

At the end of every step t, `Market.end_step()` calls `PublicState.update(book, buy_volume, sell_volume, last, flow_buy, flow_sell, step_trades)`. `PublicState` (`order_book/public_state.py`) is the **only** market state object that agents, detectors, and the executor are allowed to read.

Let b_t, a_t be the best bid and best ask at the end of step t. If both sides are non-empty, $m_t = (b_t + a_t)/2$ and $\text{spread}_t = a_t - b_t$. If one side is empty, m_t falls back to the last trade price p_{last} (or the previous mid m_{t-1} if no trade has occurred) and $\text{spread}_t = 0$. Let $\Delta m_t = m_t - m_{t-1}$ be the one-step mid change, and $V_t = V_t^{\text{buy}} + V_t^{\text{sell}}$ be total step volume.

1. Fast and slow EWMAs. With fast span $\tau_f = 10$ ($\alpha_f = 2/(\tau_f + 1) = 2/11$) and slow span $\tau_s = 100$ ($\alpha_s = 2/(\tau_s + 1) = 2/101$), `PublicState` updates six exponential weighted moving averages in $O(1)$:

$$\bar{m}_t^f = \bar{m}_{t-1}^f + \alpha_f(m_t - \bar{m}_{t-1}^f), \qquad \bar{m}_t^s = \bar{m}_{t-1}^s + \alpha_s(m_t - \bar{m}_{t-1}^s), \tag{12}$$

$$\sigma_t^{2,f} = \sigma_{t-1}^{2,f} + \alpha_f((\Delta m_t)^2 - \sigma_{t-1}^{2,f}), \qquad \sigma_t^{2,s} = \sigma_{t-1}^{2,s} + \alpha_s((\Delta m_t)^2 - \sigma_{t-1}^{2,s}), \tag{13}$$

$$\bar{V}_t^f = \bar{V}_{t-1}^f + \alpha_f(V_t - \bar{V}_{t-1}^f), \qquad \bar{V}_t^s = \bar{V}_{t-1}^s + \alpha_s(V_t - \bar{V}_{t-1}^s). \tag{14}$$

2. Multi-scale momentum indicators. For each lookback pair $(\ell_{f,k}, \ell_{s,k}) \in \{(5, 30),\ (10, 60),\ (20, 120)\}$ ($k \in \{0, 1, 2\}$), two separate mid EMAs $E_t^{(k,f)}$ and $E_t^{(k,s)}$ with weights $2/(\ell_{f,k} + 1)$ and $2/(\ell_{s,k} + 1)$ are updated, yielding the three public momentum signals:

$$\mu_t^{(k)} = E_t^{(k,f)} - E_t^{(k,s)} \quad \text{(in ticks)}. \tag{15}$$

3. Ring-buffer order-flow imbalance. Over a rolling window of $W = 20$ steps (`flow_window`), `PublicState` maintains two fixed-length ring buffers `_buy[0..W-1]` and `_sell[0..W-1]` together with running sums $B_t = \sum_{u=t-W+1}^{t} V_u^{\text{flow,buy}}$ and $S_t = \sum_{u=t-W+1}^{t} V_u^{\text{flow,sell}}$ (evicting slot $t \bmod W$ and adding the new step's visible aggressive flow in $O(1)$). The public order-flow imbalance is:

$$I_t = \begin{cases} \dfrac{B_t - S_t}{B_t + S_t} \in [-1, 1] & \text{if } B_t + S_t > 0, \\ 0.0 & \text{if } B_t + S_t = 0. \end{cases} \tag{16}$$

4. Attention multiplier (endogenous activity clustering). Once the step counter reaches $\tau_s = 100$ (`warmed_up = True`), `PublicState` computes a bounded activity multiplier that rises when fast volatility or fast volume exceeds its slow baseline:

$$a_t = \min\left(3.0,\ \max\left(0.5,\ 1.0 + 0.5\left(\sqrt{\frac{\sigma_t^{2,f}}{\sigma_t^{2,s}}} - 1\right) + 0.5\left(\frac{\bar{V}_t^f}{\bar{V}_t^s} - 1\right)\right)\right), \tag{17}$$

with $a_t = 1.0$ before warm-up. This creates realistic volatility and volume clustering without hard-coding regime switches.

6 Randomness Architecture and Common Random Numbers (`order_book.simulator`)

To measure small leakage effects (~ 0.3 bps) buried inside background price fluctuations whose standard deviation over 2 000 steps is ~ 10 bps, the simulator must support **paired counterfactuals with Common Random Numbers (CRN)**.

In `Simulator.__init__(seed, config, analysis)`, a single root `np.random.SeedSequence(seed)` spawns a sequence of statistically independent PCG64 streams via `new_stream()`:

$$\underbrace{\text{Stream } 0}_{\text{Scheduler}} \to \underbrace{\text{Stream } 1}_{\text{Fundamental } v_t} \to \underbrace{\text{Streams } 2..5}_{\text{Market Makers}} \to \underbrace{\text{Stream } 6}_{\text{NoiseFlow}} \to \underbrace{\text{Streams } 7..6+n_{\text{inst}}}_{\text{Institutions}} \to \underbrace{\text{Next Stream}}_{\text{Speculators}} \to \underbrace{\text{Next}}_{\text{Anticipators}} \to \underbrace{\text{Next}}_{\text{Externals}}.$$

This design provides four mathematical guarantees verified in `tests/test_foundations.py`:

1. **Bitwise determinism:** Running `Simulator(seed, cfg)` twice produces identical order IDs, trade prices, quantities, and mid paths down to the last bit.
2. **Snapshot/restore equivalence:** `blob = sim.snapshot()` (`pickle.dumps`) followed by `Simulator.restore(blob)` preserves the exact internal state of every PCG64 bit generator.
3. **In-place reseeding:** `sim.reseed(new_seed)` spawns a new `SeedSequence(new_seed)` and overwrites `stream.bit_generator.state` in place for every existing stream, so every agent holding a reference to its stream immediately follows the new seed.
4. **Counterfactual pairing:** Because external agents (`add_external`) are appended at the end of the stream list and act outside the scheduler's permutation, two runs on the same seed (e.g. visible vs. invisible executor, or with vs. without an injected metaorder) share the exact same random numbers for the hidden fundamental value, the scheduler's permutation, and every background agent up to the step where the book state itself first differs.

That last guarantee is the weak one: after the first difference, a state-dependent number of draws shifts each stream for good. `SimConfig(rng="counter")` replaces these streams with draws addressed by agent, step and purpose, and the default stays as described here so that all stored results reproduce (§18).

7 Mathematical Specification of Every Background Agent (`order_book.agents`)

All agents inherit from `Agent(cid, market, rng, cash, shares)` in `order_book/agents/base.py` (except `Speculators`, which vectorises $n = 40$ accounts inside one object for speed). Below are the exact mathematical equations implemented in each file.

7.1 Hidden Fundamental Value (`agents/fundamental.py`)

In the default *shared-value* mode (`FundamentalConfig.private = False`), the exogenous fundamental value v_t (initialised at $v_0 = m_0 = 10\,000$ ticks) evolves at the start of step t as an Ornstein–Uhlenbeck / Gaussian random walk:

$$v_t = v_{t-1} + \rho_v(v_0 - v_{t-1}) + \sigma_v \varepsilon_t, \qquad \varepsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1), \tag{18}$$

with default $\sigma_v = 0.3$ ticks/step and $\rho_v = 0.0$.

In the alternative *private-valuations* mode (`private = True`), there is no common anchor: each speculator $i \in \{1, \ldots, n_{\text{spec}}\}$ maintains its own private valuation $v_{i,t}$ that adapts toward the previous step's public mid-price m_{t-1}:

$$v_{i,t} = v_{i,t-1} + \kappa_a(m_{t-1} - v_{i,t-1}) + \sigma_v \varepsilon_{i,t}, \qquad \varepsilon_{i,t} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1), \tag{19}$$

with default $\kappa_a = 0.01$. Notice that in private mode `FundamentalValue.advance()` still draws one normal variate from its own stream each step so that stream alignment is preserved.

7.2 Zero-Intelligence Noise Flow (`agents/noise.py`)

`NoiseFlow` represents retail and liquidity-motivated background traders using a single pooled account (endowment 6 000 shares and 6 000 $\times$ m_0 cash). On each step t:

1. **Random cancellation:** Let R_t be the list of currently resting order IDs owned by the noise pool. Draw $K_t \sim \text{Binomial}(|R_t|,\ p_{\text{cancel}})$ with $p_{\text{cancel}} = 0.015$, sample K_t order IDs uniformly without replacement from R_t, and cancel them.
2. **Attention-modulated Poisson arrivals:** Draw the number of new orders
$$N_t \sim \text{Poisson}(\lambda_{\text{noise}}\, a_t), \qquad \lambda_{\text{noise}} = 2.5, \tag{20}$$
where $a_t \in [0.5, 3.0]$ is the public attention multiplier.
3. **Wealth-balanced side probability:** To prevent the noise pool from drifting into cash or share exhaustion over long simulations, compute its current cash share of mark-to-market wealth:
$$c_t = \frac{\text{cash}_t}{\text{cash}_t + \text{shares}_t \cdot m_t}, \qquad p_{\text{bid},t} = 0.5 + w_{\text{bal}}(c_t - 0.5), \qquad w_{\text{bal}} = 0.5. \tag{21}$$
If cash is high ($c_t > 0.5$), $p_{\text{bid},t} > 0.5$ so the pool buys more often; if shares are high ($c_t < 0.5$), it sells more often.
4. **Order size, type, and price placement:** For each of the $i = 1, \ldots, N_t$ orders, draw size $q_i \sim \text{Uniform}\{1, 2, \ldots, q_{\max}\}$ ($q_{\max} = 12$), type selector $u_i \sim \text{Uniform}(0, 1)$, and price offset $d_i \sim \text{Geometric}(p_{\text{geom}} = \frac{1}{\bar{d}+1})$ on $\{1, 2, \ldots\}$ with $\bar{d} = 3$ (`mean_offset`):
   - If $u_i < f_{\text{mkt}} = 0.15$: submit a **market order** of half size $\lfloor (q_i + 1)/2 \rfloor$ (reflecting the empirical fact that aggressive takers use smaller order sizes than passive liquidity providers).
   - Else if $u_i < f_{\text{mkt}} + f_{\text{cross}} = 0.20$: submit a **marketable limit order** of size q_i priced right at the opposite best ($p = a_t$ for a bid, $p = b_t$ for an ask).
   - Else (80% of orders): submit a **passive limit order** of size q_i placed d_i ticks behind the opposite best:
$$p_i = \begin{cases} a_t - d_i & \text{if buying (`"bid"`),} \\ b_t + d_i & \text{if selling (`"ask"`).} \end{cases} \tag{22}$$

7.3 Fundamentalists and Chartists (`agents/speculators.py`)

`Speculators` models $n = 40$ boundedly rational traders (endowment 400 shares each, position cap $P_{\max} = 400$ shares relative to endowment). At initialization, each member $i \in \{0, \ldots, 39\}$ draws:

- a permanent private valuation bias $\beta_i \sim \mathcal{N}(0, \sigma^2_{\text{bias}})$ with $\sigma_{\text{bias}} = 3.0$ ticks;
- an initial strategy mode $M_{i,0} \in \{0$ (fundamentalist), 1 (chartist)$\}$ uniformly;
- a momentum lookback index $k_i \in \{0, 1, 2\}$ uniformly (corresponding to EMA pairs $(5, 30), (10, 60), (20, 120)$).

Step activation. On each step t, each speculator independently becomes active with probability $p_{\text{act}} = 0.05$ (on average 2 active speculators per step). An active speculator i first cancels all of its resting orders (`m.cancel_all(cid)`) and computes its net inventory $x_{i,t} = \text{shares}_{i,t} - \text{shares}_{i,0}$.

Mode $M_{i,t} = 0$: Fundamentalist. Forms a noisy perception of fundamental value:

$$\hat{v}_{i,t} = \tilde{v}_{i,t} + \beta_i + \eta_{i,t}, \qquad \eta_{i,t} \sim \mathcal{N}(0, \sigma^2_{\text{obs}}), \quad \sigma_{\text{obs}} = 1.0 \text{ tick}, \tag{23}$$

where $\tilde{v}_{i,t} = v_t$ in shared-value mode and $v_{i,t}$ in private-valuation mode. Define the mispricing gap $g_{i,t} = \hat{v}_{i,t} - m_t$.

- If $|g_{i,t}| < \theta_f = 2.0$ ticks: do nothing.
- Otherwise, trade in direction side = `"bid"` if $g_{i,t} > 0$ else `"ask"`, with desired quantity capped by both $q_{f,\max} = 20$ and remaining position room:

$$q = \min\big(q_{f,\max},\ \max(1, \mathrm{round}(|g_{i,t}| - \theta_f)),\ P_{\max} - \mathrm{sign}(g_{i,t})\, x_{i,t}\big). \tag{24}$$

- If $|g_{i,t}| \geq g_{\mathrm{aggr}} = 8.0$ ticks, submit a **market order** for q. Otherwise submit a **passive limit order** priced at $\min(\lfloor \hat{v}_{i,t} - \theta_f \rfloor, a_t - 1)$ for a buy or $\max(\lceil \hat{v}_{i,t} + \theta_f \rceil, b_t + 1)$ for a sell.

Mode $M_{i,t} = 1$: Chartist (trend follower). Reads its assigned momentum signal $\mu = \mu_t^{(k_i)}$:

- **Exit rule:** If the trader holds a non-zero position ($x_{i,t} \neq 0$) and the trend has either reversed ($x_{i,t} \cdot \mu < 0$) or faded ($|\mu| < 0.25\,\theta_c$), it unwinds toward zero by submitting a market order of size $\min(|x_{i,t}|, q_{c,\max})$ in the opposite direction.
- **Entry rule:** If $|\mu| \geq \theta_c = 6.0$ ticks, it submits a market order in the direction of $\mathrm{sign}(\mu)$ with size

$$q = \min\big(q_{c,\max},\ \max(1, \mathrm{round}(s_c(|\mu| - \theta_c))),\ P_{\max} - \mathrm{sign}(\mu)\, x_{i,t}\big), \tag{25}$$

  where $q_{c,\max} = 10$ lots and $s_c = 2.0$ lots/tick.

Brock–Hommes evolutionary mode switching. Every $T_{\mathrm{switch}} = 50$ steps, let $W_{i,t} = \mathrm{cash}_{i,t} + \mathrm{shares}_{i,t} m_t$ be each speculator's mark-to-market wealth and $r_{i,t} = 100 \times (W_{i,t} - W_{i,t-50})/W_{i,t-50}$ be its percentage return over the 50-step block. The fitness F_m of each mode $m \in \{0, 1\}$ is updated via an EWMA with memory $\lambda_F = 0.1$:

$$F_m \leftarrow F_m + \lambda_F(\bar{r}_m - F_m), \qquad \bar{r}_m = \frac{1}{|\{i : M_i = m\}|} \sum_{i : M_i = m} r_{i,t}, \tag{26}$$

and every speculator re-draws its mode $M_{i,t+1} = 1$ (chartist) with logistic probability

$$\mathbb{P}(M_{i,t+1} = 1) = \frac{1}{1 + \exp(-\beta_{\mathrm{BH}}(F_1 - F_0))}, \qquad \beta_{\mathrm{BH}} = 2.0. \tag{27}$$

7.4 Institutional Traders (`agents/institutional.py`)

The population contains n_{inst} background institutional traders (4 in full-cover regimes, 1 in `low_cover`, 0 in `no_cover`). Their economic role is crucial: because they execute large parent orders using standard industry schedules, they generate legitimate one-sided order flow that creates **false positives** for detectors and provides **statistical cover** for the RL executor.

Metaorder arrival and parameters. While an institutional trader is idle, it starts a new metaorder on step t with probability $\lambda_{\mathrm{inst}} = 5 \times 10^{-4}$ (mean idle time 2 000 steps). Upon arrival:

1. **Inventory-reverting side choice:** With normalized inventory $x_t = (\mathrm{shares}_t - \mathrm{shares}_0)/\mathrm{shares}_0$, the metaorder is a buy (`"bid"`) with probability

$$p_{\mathrm{bid}} = \frac{1}{2} - \frac{1}{2}\tanh(\pi_I x_t), \qquad \pi_I = 2.0. \tag{28}$$

2. **Log-normal size:** Draw $Z \sim \mathcal{N}(0, 1)$ and set

$$Q = \min\big(Q_{\max},\ \max(1, \mathrm{round}(Q_{\mathrm{med}} \exp(\sigma_Q Z))),\ \text{affordable lots}\big), \tag{29}$$

   with $Q_{\mathrm{med}} = 600$ lots, $\sigma_Q = 0.7$, and $Q_{\max} = 3\,000$ lots.

3. **Participation rate and duration:** Draw target participation $\pi \sim \text{Uniform}(0.02, 0.20)$. Using the public slow volume EWMA $\bar{V}_t^s$ (or $V_{\text{fallback}} = 5.0$ lots/step before warm-up), the planned horizon is

$$T = \text{clamp}\Big(\text{round}\Big(\frac{Q}{\pi\, \bar{V}_t^s}\Big),\ 100,\ 4\,000\Big) \text{ steps.} \tag{30}$$

4. **Execution style:** Drawn from the 5 styles (`twap_market`, `twap_random`, `twap_passive`, `pov`, `front_loaded`) with child decision interval $\Delta = 5$ steps. At $t = t_{\text{start}} + T$, any remaining unfilled quantity is forced through by market order.

7.5 Avellaneda–Stoikov Market Makers (`agents/market_maker.py`)

Each of the $n_{\text{mm}} = 4$ market makers quotes both a bid and an ask on every step, managing its inventory $I_t = \text{shares}_t - \text{shares}_0$ within $[-I_{\max}, I_{\max}]$ ($I_{\max} = 200$).

1. Volatility fragility and pull cooldown. Let the fast-to-slow volatility ratio be $\varrho_t = \sqrt{\sigma_t^{2,f}/\sigma_t^{2,s}}$. If $\varrho_t > \varrho_{\text{pull}} = 3.0$, the maker enters a pull state for $T_{\text{cool}} = 20$ steps (pulled_until $= t + 20$), cancelling both quotes immediately and staying out of the book until $t \geq$ pulled_until. Otherwise, its base quote size is scaled by the fragility factor:

$$f_{\text{frag},t} = \max\big(0.0,\ 1.0 - \phi\, \max(\varrho_t - 1.0, 0.0)\big), \qquad \phi = 0.5. \tag{31}$$

2. Avellaneda–Stoikov reservation price and half-spread. With effective variance $\sigma_t^2 = \max(\sigma_t^{2,f}, \sigma_{\min}^2)$ ($\sigma_{\min}^2 = 0.25$), risk aversion $\gamma = 0.003$, liquidity parameter $\kappa = 1.0$, horizon $H = 50$, and cap $\bar{s} = 10.0$ ticks:

$$\text{Risk term:} \quad R_t = \gamma\, \sigma_t^2\, H, \tag{32}$$

$$\text{Reservation price:} \quad r_t = m_t - \text{clamp}(I_t\, R_t,\ -\bar{s},\ +\bar{s}), \tag{33}$$

$$\text{Half-spread:} \quad h_t = \text{clamp}\Big(\frac{1}{2}\Big(R_t + \frac{2}{\gamma}\ln\big(1 + \frac{\gamma}{\kappa}\big)\Big),\ 1.0,\ \bar{s}\Big). \tag{34}$$

3. Detector-driven defensive response (the source of leakage cost). If the market maker is wired to a detector (`cfg.detector is not None`), it reads the detector's signed signal $s_t \in [-1, 1]$. When $s_t > 0$, a buy metaorder is suspected, so the **pressured side** is the `"ask"` (buyers lift asks); when $s_t < 0$, the pressured side is the `"bid"`:

$$r_t \leftarrow r_t + \delta_{\text{skew}}\, s_t, \qquad (\delta_{\text{skew}} = 4.0 \text{ ticks}), \tag{35}$$

$$\omega_{\text{pressured}} = \delta_{\text{widen}}\, |s_t|, \quad \omega_{\text{other}} = 0, \qquad (\delta_{\text{widen}} = 3.0 \text{ ticks}), \tag{36}$$

$$\text{scale}_{\text{pressured}} = f_{\text{frag},t} \cdot \max(0.0,\ 1.0 - \delta_{\text{cut}}\, |s_t|), \qquad (\delta_{\text{cut}} = 0.7), \tag{37}$$

$$\text{scale}_{\text{other}} = f_{\text{frag},t}. \tag{38}$$

Target quote prices are then $p_{\text{bid}} = r_t - h_t - \omega_{\text{bid}}$ and $p_{\text{ask}} = \max(r_t + h_t + \omega_{\text{ask}},\ p_{\text{bid}} + 2)$, clamped to be strictly post-only against all other participants' best quotes via `book.best_excluding(side, own_oid)`.

4. Queue-position preservation. A resting quote on side $S \in \{\text{bid}, \text{ask}\}$ is kept untouched (preserving its FIFO queue priority!) if and only if $|p_{\text{resting}} - p_{\text{target}}| < 1.0$ tick and its remaining quantity satisfies $0.5\, q_{\text{target}} \leq q_{\text{resting}} \leq q_{\text{target}}$. Otherwise it is cancelled and replaced with a freshly drawn base size $q_{\text{base}} \sim \text{Uniform}\{5, \ldots, 20\}$ multiplied by scale_S.

7.6 Order Anticipator (`agents/anticipator.py`)

Present under adversary level `A4`, `OrderAnticipator` reads s_t = detector.signal and trades aggressively with market orders:

- If $|s_t| \geq \theta_A = 0.5$: increment `confident_steps` and trade in direction $d = \text{sign}(s_t)$ (or $-d$ for the first `switch_after` $= 50$ steps if `contrarian_first=True`) with market order quantity

$$q = \min\Big(c_{\max},\ \text{round}\Big(\varphi\,\frac{|s_t| - \theta_A}{1 - \theta_A}\Big),\ P_{\max} - d \cdot x_t\Big), \qquad (\varphi = 2.0,\ c_{\max} = 10,\ P_{\max} = 200). \tag{39}$$

- If $|s_t| < \theta_{\text{fade}} = 0.2$: reset `confident_steps` $= 0$ and unwind any non-zero inventory x_t by submitting a market order of size $\min(c_{\max}, |x_t|)$ in direction $-\text{sign}(x_t)$.

8 Simulator Validation: Stylised Facts and Square-Root Impact (Phase 1)

Before any execution algorithm or detector is evaluated, the background market (`A0`, no executor) must reproduce the empirical regularities ("stylised facts") of real limit order books and the concave square-root market-impact law. This validation is implemented in `order_book/validation.py` and `order_book/impact.py` and run via `scripts/validate.py` and `scripts/impact.py`.

8.1 Paired-Injection Square-Root Impact Methodology (`order_book.impact`)

To measure the mechanical impact law $\Delta m \propto (Q/V)^{\delta}$ cleanly without the noise of the background price random walk ($\sigma\sqrt{T} \approx 8$–12 ticks over $T = 2\,000$ steps), `impact.py` uses Common Random Numbers (§6):

1. For each seed, run a reference background simulation after a 2 000-step burn-in for $T_{\text{ref}} + T_{\text{after}} = 2\,500$ steps and record its mid-price path m_t^{bg} and mean step volume $\bar{V}$.
2. For each participation fraction $f \in \{0.01, 0.02, 0.05, 0.10, 0.20\}$, run the identical seed with an external `InstitutionalTrader` (`add_external`) that injects a single buy metaorder of size $Q = f \cdot \bar{V} \cdot T_{\text{ref}}$ executed by `twap_market` over $T_{\text{ref}} = 2\,000$ steps, yielding mid path $m_t^{(f)}$.
3. The paired impact path $\Delta m_t^{(f)} = m_t^{(f)} - m_t^{\text{bg}}$ cancels the background price drift. From $\Delta m_t^{(f)}$ we read impact at completion ($t = T_{\text{ref}}$), peak impact ($\max_{t \leq T_{\text{ref}}} \Delta m_t^{(f)}$), and residual impact 500 steps after completion ($t = T_{\text{ref}} + 500$), and fit the power-law exponent δ by OLS of log(impact) on $\log(f)$.

8.2 Phase 1 Validation Results (`outputs/validation_default.json`)

Table 2 reports the measured validation metrics on 5 seeds $\times$ 30 000 steps (after 2 000 burn-in steps) against the research plan's Phase 1 exit gate targets. Every target passes.

9 Specification of All Detectors (`order_book.detectors`)

Every detector implements `Detector.update(public: PublicState) -> float` and exposes `signal` $\in [-1, 1]$, `p_buy` $\in [0, 1]$, and `p_sell` $\in [0, 1]$.

Stylised Fact / Check	Plan Target (§4.2)	Measured Value (5 seeds × 30k steps)
Book & account integrity	Never crossed; exact cash/share conservation	Asserted every step in tests: **PASS**
Liquidity & spread	Median spread 1–3 ticks; empty side $< 0.1\%$	Median spread **1.00** tick; empty side **0.000**%: **PASS**
Return autocorrelation	$\lvert ac(\text{lag})\rvert < 0.05$ for lags ≥ 5 (50+ steps)	$\max_{\text{lag}\geq 5} \lvert ac\rvert = \mathbf{0.044}$: **PASS**
Heavy return tails	Kurtosis $\gg 3$ at $\Delta t = 1$, decaying toward 3	**12.7** ($\Delta t = 1$) $\rightarrow$ **6.9** ($\Delta t = 100$): **PASS**
Volatility clustering	$ac(\lvert r\rvert) > 0$, slow decay over lags 1..200	**0.20** (lag 1), **0.15** (lag 20), **0.09** (lag 100): **PASS**
Order-flow memory	Trade-sign $ac > 0$, slow decay	**0.46** (lag 1), **0.09** (lag 10), **0.04** (lag 100): **PASS**
Square-root impact (δ)	$\delta \in [0.4, 0.7]$	Paired: **0.69** at end, **0.64** at peak ($0.8 \rightarrow 6.0$ ticks for $f = 0.01 \rightarrow 0.20$); natural ($n = 181$): **0.60** end, **0.54** peak: **PASS**
Impact relaxation	Partial reversion after $t = T$	Paired: end is **52**% of peak, **42**% remains at $T + 500$: **PASS**
Budget stability	$< 1\%$ budget rejects, balanced	**0** rejects in $\approx 190\,000$ trades: **PASS**
Simulation speed	$\geq 10\,000$ steps/s (1 core)	**14 800** steps/s (analysis), **15 200** steps/s (training): **PASS**

Table 2: Phase 1 simulator validation targets and measured values (`outputs/validation_default.json`).

9.1 Level A1: Rule-Based Flow Imbalance Detector (`detectors/flow.py`)

Maintains an EWMA M_t of the 20-step signed order-flow imbalance I_t (16) with memory span $\tau_d = 10$ ($\alpha_d = 2/11$):

$$M_t = M_{t-1} + \frac{2}{\tau_d + 1}(I_t - M_{t-1}), \qquad s_t = \text{sign}(M_t) \cdot \min\Big(1.0,\ \max\Big(0.0,\ \frac{|M_t| - \theta_d}{1 - \theta_d}\Big)\Big), \tag{40}$$

where $\theta_d \in \{0.20\ \text{(lenient)},\ 0.30\ \text{(medium)},\ 0.45\ \text{(strict)}\}$.

9.2 Public Feature Tracker (`detectors/features.py`)

`FeatureTracker` maintains a 200-step deque of per-step flow tuples $(V_t^{\text{buy}}, V_t^{\text{sell}}, N_t^{\text{trades}}, N_t^{\text{buy_trades}})$, a volume-clock deque of the most recent aggressive trades summing to at most $V_{\text{clock}} = 100$ lots, a 51-step mid-price deque, and the current same-sign aggressive trade run (sign, length, and last 10 trade sizes). On each step it outputs the 10-dimensional feature vector $\mathbf{f}_t \in \mathbb{R}^{10}$:

Index	Name	Exact Formula
f_0	`imbalance_20`	20-step volume imbalance $I_t^{(20)} = (B_{20} - S_{20})/(B_{20} + S_{20})$
f_1	`imbalance_50`	50-step volume imbalance $I_t^{(50)} = (B_{50} - S_{50})/(B_{50} + S_{50})$
f_2	`imbalance_200`	200-step volume imbalance $I_t^{(200)} = (B_{200} - S_{200})/(B_{200} + S_{200})$
f_3	`imbalance_volume_clock`	Signed volume imbalance over the last 100 traded lots
f_4	`signed_run`	$\text{run_sign}_t \cdot \min(\text{run_length}_t, 20)/10 \in [-2, 2]$
f_5	`regularity`	$\max(0, 1 - \text{std}(\text{run_sizes})/\text{mean}(\text{run_sizes}))$ if $\text{run_length}_t \geq 3$ else 0
f_6	`trade_rate_ratio`	(trades in last 20 steps/20)/(trades in last 200 steps/200)
f_7	`normalised_return_50`	$(m_t - m_{t-50})/\sqrt{\max(\sigma_t^{2,f}, 10^{-6}) \cdot 50}$
f_8	`spread`	Current bid–ask spread $a_t - b_t$ (ticks)
f_9	`count_imbalance_20`	20-step trade-count imbalance $(N_{20}^{\text{buy}} - N_{20}^{\text{sell}})/N_{20}^{\text{total}}$

9.3 Level A2: Four Held-Out Statistical Detectors (`detectors/statistical.py`)

1. **`RunsDetector`:** Fires on persistent streaks of same-sign aggressive trades:

$$s_t = \text{run_sign}_t \cdot \text{clip}\Big(\frac{\text{run_length}_t - 4}{6},\ 0,\ 1\Big). \tag{41}$$

2. **`CusumDetector`:** Two-sided Page's CUSUM with restart on non-overlapping 20-step blocks. At the end of each 20-step block with block imbalance x_b:

$$S_b^+ = \max(0, S_{b-1}^+ + x_b - k), \qquad S_b^- = \max(0, S_{b-1}^- - x_b - k), \qquad (k = 0.1). \tag{42}$$

   When $S_b^+ > h = 1.5$ or $S_b^- > h = 1.5$, an alarm is triggered ($\text{alarm_sign} = \pm 1$), both S_b^+, S_b^- reset to 0, and the alarm signal decays linearly over $H_{\text{hold}} = 10$ blocks: $s_t = \text{alarm_sign} \cdot \max(0, 1 - b_{\text{since}}/10)$ (or $(S_b^+ - S_b^-)/h$ if larger in magnitude).
3. **`VolumeClockDetector`:** Applies the A1 dead-zone threshold ($\theta = 0.3$) to the 100-lot volume-clock imbalance f_3 instead of wall-clock time.
4. **`RegularityDetector` (miniature Goliath–Gebbie metaorder reconstruction):** Detects child-order splitting that is both persistent ($\text{run_length}_t \geq 3$) and suspiciously uniform in slice size:

$$s_t = \mathbb{I}(\text{run_length}_t \geq 3) \cdot \text{run_sign}_t \cdot f_{5,t} \cdot \min\Big(1.0,\ \frac{\text{run_length}_t}{6}\Big). \tag{43}$$

9.4 Level A3: Learned Multinomial Logistic Classifier (`detectors/learned.py`)

Trained offline by `scripts/train_detector.py` on background runs (40 training seeds × 20 000 steps, 8 validation seeds) where only background institutions trade. Each step t is labelled from `MetaorderLog` as $y_t \in \{0$ (none), 1 (net buy active), 2 (net sell active)$\}$. Standardised features $\tilde{\mathbf{f}}_t = (\mathbf{f}_t - \boldsymbol{\mu})/\boldsymbol{\sigma}$ feed a 3-class linear layer $\mathbf{z}_t = \tilde{\mathbf{f}}_t \mathbf{W} + \mathbf{b} \in \mathbb{R}^3$ with softmax probabilities $\mathbf{p}_t = \text{softmax}(\mathbf{z}_t)$ and signed signal $s_t = p_{t,1} - p_{t,2}$. Fitted weights (saved in `outputs/detectors/learned.npz`) place positive weight on multi-horizon imbalances (f_0, f_1, f_2, f_3), run length (f_4), size regularity (f_5), spread (f_8), and count imbalance (f_9).

Part III
Phase 2: Execution Baselines, Counterfactual Methodology, and Empirical Results

10 Execution Task and Baselines (`order_book.execution`)

An `ExecutionTask` (`order_book/execution/task.py`) specifies:

- `side` $\in$ {`"bid"`, `"ask"`} (fixed by seed parity: even seeds buy, odd seeds sell);
- `size_fraction` f: target parent size $Q = \max(1, \text{round}(f\,\bar{V}\,T))$, where $\bar{V}$ is the mean traded volume per step measured over the 2 000-step burn-in on that seed;
- `horizon` $T = 2\,000$ steps and `decision_interval` $N = 5$ steps (400 decision points per episode);
- Hard risk limits enforced by `ExecutorAgent._apply`: at most `max_resting_orders` $= 2$ simultaneous resting orders, marketable limit price protection of `max_through_ticks` $= 5$ ticks through the opposite best, and mandatory market-order forced completion at $t \geq t_{\text{start}} + T$ (with up to `grace` $= 50$ steps if the book is thin).

Table 3 summarises the mathematical decision rules of all twelve baseline strategies in `order_book/execution/strategies.py`, where $R_t = Q - X_t$ is remaining quantity, $S_t = \max(1, \lceil (T - t)/N \rceil)$ is decisions left, and X_t is quantity executed so far.

Calibration under A0 (`outputs/calibration.json`). Running `scripts/calibrate.py` on training seeds under `A0` yields the fitted structural parameters used by the model-based strategies:

$$\sigma = 0.1709 \text{ ticks/step}, \qquad \eta = 0.0812 \text{ ticks/(lot/step)}, \qquad \gamma_{\text{perm}} = 0.00368 \text{ ticks/lot},$$
$$\rho = 0.00253 \text{ step}^{-1}, \qquad \text{depth}_5 = 229.4 \text{ lots},$$

and 5-step passive fill probabilities $p_d = (0.160,\ 0.0063,\ 0.0004,\ 0.0001)$ at distances $d \in \{0, 1, 2, 3\}$ ticks behind own best.

11 Phase 2 Experimental Results: E2, E3, E7, and E7b

All Phase 2 experiments were executed on held-out test seeds 1000..1019 ($n = 20$ paired episodes per cell, even seeds buying, odd seeds selling, $T = 2\,000$, burn-in 2 000).

11.1 Experiments E2 and E3: The Cover Finding (`outputs/e2`, `e3`)

In the default full-cover market (`deep_cover`: 4 background institutions, full liquidity), E2 measured the ROC AUC of all 8 detectors at distinguishing steps when the executor is active from steps when it is not, and E3 measured the counterfactual leakage $\text{IS}_{\text{visible}} - \text{IS}_{\text{invisible}}$ across all 12 strategies, 3 sizes $Q/V \in \{0.05, 0.10, 0.20\}$, and 3 adversary levels (`A0`, `A1-medium`, `A4`; 4 380 total episodes).

Interpretation of E2 and E3.

1. **Sanity check passes:** Under `A0` (no detecting agents), leakage is identically 0.00 bps to machine precision across every strategy and size.
2. **Detection channel works when triggered:** Strategies that dump large aggressive volume in a short window (`immediate` at $Q/V = 0.05, 0.10$ and front-loaded `almgren_chriss_1e-05` at $Q/V = 0.20$) suffer statistically significant leakage (+0.48 bps under A1, amplified to +1.68 bps and +4.84 bps by the order anticipator in A4).

Strategy Name	Family	Mathematical Decision Rule at Decision Step t
`immediate`	Classical	Submit market order $q_{\text{mkt}} = R_t$ at $t = 0$.
`twap_market`	Classical	Submit market order $q_{\text{mkt}} = \lceil R_t / S_t \rceil$.
`twap_passive`	Classical	Cancel resting; cross catch-up $c_t = \max(0, \text{target}_{t-N} - X_t)$ by market order; post $q_{\text{lim}} = \lceil (R_t - c_t)/S_t \rceil$ at own best.
`twap_random`	Classical	Skip decision with prob $p_{\text{skip}} = 0.30$ (if $S_t > 1$); else submit $q_{\text{mkt}} = \lceil (R_t/S_t) \cdot U(0.5, 1.5) \rceil$.
`pov`	Classical	If $t < 0.9T$, submit $q_{\text{mkt}} = \max(0, \min(Q, \lfloor 0.10\, V^{\text{pub}}_{0:t} \rfloor) - X_t)$; else TWAP market catch-up.
`almgren_chriss_1e-07`	Model-based	Schedule $X^*(t) = Q\big(1 - \frac{\sinh(\kappa(T-t))}{\sinh(\kappa T)}\big)$ with $\kappa = \sqrt{\lambda\sigma^2/\eta}$, $\lambda = 10^{-7}$ ($\kappa T \approx 0.42$); passive-with-catch-up tactic.
`almgren_chriss_1e-06`	Model-based	Same with $\lambda = 10^{-6}$ ($\kappa T \approx 1.33$, moderately front-loaded).
`almgren_chriss_1e-05`	Model-based	Same with $\lambda = 10^{-5}$ ($\kappa T \approx 4.20$, strongly front-loaded).
`obizhaeva_wang`	Model-based	Transient impact resilience ρ: initial block $x_0 = Q/(\rho T + 2)$ at $t = 0$, constant market rate ρx_0 for $0 < t < T$, terminal block at T.
`limit_market`	Model-based	Pick deepest offset $d \in \{0, 1, 2, 3\}$ with $p_d S_t \geq 1.5$; allow schedule slack s_{slice}/p_d; keep resting order if price unchanged (queue priority); market catch-up over last 2 decisions.
`footprint`	Stealth	Base $b = R_t/S_t \cdot U(0.6, 1.4)$; if own_flow_share$_t > 0.30$, scale $\times 0.25$ (back off); if sign $\cdot I_t > 0.20$, scale $\times 1.5$ (hide in natural flow); split 50% market / 50% passive; TWAP market over last 15%.
`mimic`	Stealth	At episode start, draw one of the 5 institutional styles uniformly and execute with institutional parameters so the executor is statistically indistinguishable from an institution.

Table 3: The twelve baseline execution strategies in `order_book/execution/strategies.py`.

Strategy	Q/V	Adversary	IS_{vis}	IS_{invis}	Leakage [95% CI] (bps)	$p > 0$	p_{Holm}
`immediate`	0.05	A1-medium	+4.57	+4.09	+0.48 [+0.06, +1.10]	**0.000**	**0.000**
`immediate`	0.05	A4	+8.47	+6.80	+1.68 [+0.10, +4.37]	**0.000**	**0.000**
`immediate`	0.10	A4	+27.49	+22.65	+4.84 [+0.81, +9.46]	**0.006**	0.630
`almgren_chriss_1e-05`	0.20	A4	+3.98	+3.53	+0.45 [+0.16, +0.76]	**0.001**	0.053
`twap_market`	0.05	A1-medium	+1.10	+1.12	−0.02 [−0.34, +0.28]	0.551	1.000
`twap_market`	0.10	A1-medium	+1.61	+1.42	+0.20 [−0.07, +0.51]	0.074	1.000
`twap_market`	0.10	A4	+1.90	+1.70	+0.20 [−0.11, +0.55]	0.114	1.000
`twap_market`	0.20	A1-medium	+2.67	+2.55	+0.12 [−0.22, +0.52]	0.288	1.000
`twap_passive`	0.20	A1-medium	+2.39	+2.19	+0.20 [−0.05, +0.48]	0.068	1.000
`footprint`	0.20	A1-medium	+2.36	+2.41	−0.05 [−0.44, +0.29]	0.595	1.000
`mimic`	0.20	A1-medium	+3.19	+3.03	+0.16 [−0.07, +0.41]	0.092	1.000

Table 4: Selected rows from the full E3 leakage experiment (`outputs/stealth_execution/e3/summary.json`, 20 test seeds). Under `A0`, $\text{IS}_{\text{vis}} - \text{IS}_{\text{invis}} \equiv 0.00$ identically on every row.

3. **Why paced TWAP fails the E3 gate in `deep_cover`:** With 4 active background institutions, institutional metaorders are present on 55%–75% of all steps. In E2, detectors

achieve AUC ≈ 0.55–0.60 on institutional metaorders, but AUC ≈ 0.47–0.51 (chance level) at distinguishing the paced executor from institutional-only steps, with institutional false-alarm rates of 12%–27%. In short: **four background institutions provide so much natural cover that a 5%–20% TWAP is already hidden**.

11.2 Experiment E7 (`outputs/stealth_execution/e7/summary.json`): Mapping Leakage Across Market Regimes

Motivated by the E2/E3 finding, E7 (`papers/stealth_execution/scripts/e7_regimes.py`) swept six market regimes built via `presets.regime(liquidity, institutions, makers, mm_size_scale)` across $Q/V \in \{0.20, 0.40\}$ (2 160 episodes).

Regime	Liq.	Inst.	Strategy	Q/V	$\bar{V}$/stp	IS_{vis}	Leakage [95% CI] (bps)	$p > 0$	Fire
`deep_cover`	1.0	4	`twap_market`	0.20	3.6	+2.67	+0.12 [−0.22, +0.52]	0.29	0.10
`deep_cover`	1.0	4	`twap_market`	0.40	3.6	+4.93	−0.72 [−2.58, +0.31]	0.66	0.09
`low_cover`	1.0	1	`twap_market`	0.20	2.8	+2.90	**+0.31 [+0.02, +0.60]**	**0.02**	0.17
`low_cover`	1.0	1	`twap_market`	0.40	2.8	+4.91	**+0.31 [+0.01, +0.63]**	**0.02**	0.13
`low_cover`	1.0	1	`twap_passive`	0.20	2.8	+2.45	−0.07 [−0.39, +0.25]	0.68	0.19
`low_cover`	1.0	1	`twap_passive`	0.40	2.8	+4.44	−0.02 [−0.45, +0.45]	0.55	0.15
`no_cover`	1.0	0	`twap_passive`	0.20	2.7	+3.43	+0.25 [−0.02, +0.53]	0.04	0.25
`half_liquidity`	0.5	4	`twap_market`	0.20	2.3	+4.40	+3.04 [−0.51, +8.28]	0.07	0.11
`thin_low_cover`	0.5	1	`twap_market`	0.40	1.6	+4.81	+0.47 [−0.39, +1.56]	0.18	0.17
`thin_no_cover`	0.4	0	`twap_market`	0.20	1.1	+18.45	+14.38 [−3.34, +47.12]	0.32	0.32

Table 5: Key rows from the E7 regime sweep under `A1-medium` (`outputs/stealth_execution/e7/summary.json`). "Fire" is the mean $|s_t|$ of the flow detector over the executor's active window.

Key findings from E7.

1. **Hypothesis H4 confirmed:** As institutional cover falls from 4 → 1 → 0 institutions, detector firing rises monotonically (0.10 → 0.17 → 0.25 in liquid markets, and up to 0.34 in `thin_no_cover`).
2. **Liquid `low_cover` vs. thin regimes:** Cutting institutional cover from 4 to 1 while keeping the book liquid (`low_cover`) produces a clean, statistically significant leakage of +0.31 bps ($p = 0.02$) at both $Q/V = 0.20$ and 0.40, while `twap_passive` leaks ≈ 0 bps (−0.07 and −0.02 bps). By contrast, thinning the standing book (`half_liquidity`, `thin_no_cover`) causes child orders to trigger volatility-fragility quote pulls (§7.5), blowing up total IS to 18–54 bps and widening 20-seed confidence intervals to ±50 bps.

11.3 Experiment E7b (`outputs/stealth_execution/e7b/summary.json`): Detector-Strength Sweep in `low_cover`

Holding the market at the stable, liquid `low_cover` regime (liquidity = 1.0, institutions = 1), E7b (`papers/stealth_execution/scripts/e7_strength.py`) swept four detector strengths defined in `experiments.DETECTION_STRENGTHS`:

- `weak`: $\theta_d = 0.45$ (strict), 2 detecting makers, $(\delta_{\text{skew}}, \delta_{\text{widen}}, \delta_{\text{cut}}) = (4, 3, 0.7)$;
- `default`: $\theta_d = 0.30$ (medium), 2 detecting makers, $(4, 3, 0.7)$;
- `strong`: $\theta_d = 0.20$ (lenient), 4 detecting makers, $(8, 6, 0.9)$;
- `max`: $\theta_d = 0.20$ (lenient), 4 detecting makers, $(12, 10, 1.0)$.

Table 6 exhibits a textbook monotonic dose–response at $Q/V = 0.40$: as detector strength increases from `weak` → `default` → `strong` → `max`, `twap_market` leakage rises from +0.21 → +0.31 → +0.38 → +0.42 bps ($p = 0.20 \to 0.02 \to 0.01 \to 0.00$), with 100% completion throughout. Moreover, `default` is the unique sweet spot where `twap_market` leaks significantly at *both*

Strength	Strategy	Q/V	Q (lots)	IS_{vis}	IS_{invis}	Leakage [95% CI] (bps)	$p > 0$	Compl.
weak	twap_market	0.20	1 168	+2.69	+2.83	−0.15 [−0.45, +0.12]	0.85	1.00
weak	twap_market	0.40	2 335	+4.55	+4.35	+0.21 [−0.18, +0.72]	0.20	1.00
weak	twap_passive	0.20	1 168	+2.56	+2.61	−0.05 [−0.32, +0.22]	0.64	1.00
weak	twap_passive	0.40	2 335	+4.55	+4.48	+0.07 [−0.24, +0.33]	0.30	1.00
default	twap_market	0.20	1 122	+2.90	+2.59	**+0.31** [**+0.02**, **+0.60**]	**0.02**	1.00
default	twap_market	0.40	2 245	+4.91	+4.59	**+0.31** [**+0.01**, **+0.63**]	**0.02**	1.00
default	twap_passive	0.20	1 122	+2.45	+2.53	−0.07 [−0.39, +0.25]	0.68	1.00
default	twap_passive	0.40	2 245	+4.44	+4.46	−0.02 [−0.45, +0.45]	0.55	1.00
strong	twap_market	0.20	1 116	+2.66	+2.58	+0.08 [−0.15, +0.31]	0.25	1.00
strong	twap_market	0.40	2 232	+4.48	+4.10	**+0.38** [**+0.08**, **+0.67**]	**0.01**	1.00
strong	twap_passive	0.20	1 116	+2.55	+2.37	+0.18 [−0.07, +0.43]	0.08	1.00
strong	twap_passive	0.40	2 232	+4.29	+4.18	+0.11 [−0.36, +0.58]	0.32	1.00
max	twap_market	0.20	1 149	+3.06	+2.96	+0.10 [−0.21, +0.39]	0.25	1.00
max	twap_market	0.40	2 298	+5.14	+4.71	**+0.42** [**+0.17**, **+0.69**]	**0.00**	1.00
max	twap_passive	0.20	1 149	+3.09	+2.90	+0.19 [−0.07, +0.47]	0.09	1.00
max	twap_passive	0.40	2 298	+5.05	+4.45	**+0.60** [**+0.14**, **+1.18**]	**0.00**	1.00

Table 6: Complete results of the E7b detector-strength sweep in the liquid `low_cover` regime (`outputs/stealth_execution/e7b/summary.json`, 20 test seeds).

$Q/V = 0.20$ and 0.40 (+0.31 bps, $p = 0.02$) while `twap_passive` does not leak (−0.07 and −0.02 bps), creating clear economic headroom ($\sim$ 0.30–0.45 bps) for a learning executor that mixes passive placement, opportunistic timing, and footprint control. Accordingly, `low_cover + default` was selected as `phase3_config()`.

Part IV
Phase 3: Reinforcement-Learning Executor, PPO Trainer, and Sweep Results

Phase 3 (`order_book/execution/features.py`, `policy.py`, `env.py`, `ppo.py`, and `papers/stealth_execution/scripts/train_rl.py`, `papers/stealth_execution/scripts/sweep_rl.py`) embeds a learned policy directly inside the Phase 2 `ExecutorAgent`. Because `PolicyStrategy` subclasses `Strategy`, the RL agent faces the *exact same* `ExecState`, order-placement rules, resting-order caps, price protection, and horizon forced completion as every analytical baseline.

12 Mathematical Formulation of the Execution MDP

An execution episode has horizon $T = 2\,000$ simulator steps and decision interval $N = 5$ steps, so the MDP has at most $K = T/N = 400$ discrete decision stages $k \in \{0, 1, \ldots, K-1\}$ occurring at simulator steps $t_k = t_{\text{start}} + kN$ (terminating earlier if the parent order of size Q is completely filled before T).

12.1 Observation Space ($\mathcal{S} \subset \mathbb{R}^{13}$, `execution/features.py`)

At decision stage k, `features(s: ExecState)` maps the executor's public state and own progress into a 13-dimensional feature vector $\mathbf{o}_k = (o_{k,0}, \ldots, o_{k,12})^\top \in \mathbb{R}^{13}$. Every directional market feature is multiplied by sign $\in \{+1 \text{ (buy)}, -1 \text{ (sell)}\}$ so that a single policy network is invariant to whether

it is buying or selling:

Idx	Feature Name	Exact Mathematical Formula	Economic Meaning
0	`time_frac`	$\tau_k = t_{\text{elapsed}}/T \in [0,1]$	Fraction of execution horizon elapsed
1	`remain_frac`	$r_k = (Q - X_k)/Q \in [0,1]$	Fraction of parent order still unfilled
2	`schedule_gap`	$\text{clip}\big((X_k - Q\tau_k)/Q,\ -1, 1\big)$	Progress ahead of (>0) or behind (<0) linear TWAP
3	`decisions_left`	$S_k/(T/N) \in [0,1]$	Normalised remaining decision points
4	`norm_spread`	$\min(\text{spread}_k/5.0,\ 2.0)$	Current bid–ask spread (cost of crossing)
5	`signed_imbalance`	$\text{sign} \cdot I_{t_k} \in [-1,1]$	Public order-flow imbalance in own trade direction
6	`own_flow_share`	$\min\big(1,\ V_W^{\text{own,aggr}}/(B_W + S_W)\big)$	Executor's own share of 20-step aggressive flow (footprint)
7	`signed_mom_fast`	$\text{clip}(\text{sign} \cdot \mu_{t_k}^{(0)} \times 100,\ -3, 3)$	Short-term $(5, 30)$ EMA momentum in trade direction
8	`signed_mom_slow`	$\text{clip}(\text{sign} \cdot \mu_{t_k}^{(2)} \times 100,\ -3, 3)$	Long-term $(20, 120)$ EMA momentum in trade direction
9	`norm_paper_bps`	$\text{clip}\big(\frac{\text{sign} \cdot (m_{t_k} - m_{\text{arr}})}{m_{\text{arr}} \cdot 20\text{ bps}},\ -3, 3\big)$	Running mark-to-market drift from arrival mid
10	`own_resting_frac`	$\min(q_k^{\text{own,rest}}/Q,\ 2.0)$	Quantity currently resting in book as fraction of Q
11	`norm_vol`	$\min(\sqrt{\max(\sigma_{t_k}^{2,f}, 0)}/2.0,\ 3.0)$	Fast mid-price volatility (ticks/$\sqrt{\text{step}}$)
12	`norm_attention`	$\text{clip}(a_{t_k} - 1.0,\ -1.0, 2.0)$	Endogenous market activity multiplier centred at 0

Table 7: The 13-dimensional observation vector $\mathbf{o}_k$ = `features(s)` in `order_book/execution/features.py`. Strictly public data plus own inventory state.

12.2 Discrete Factored Action Space ($\mathcal{A} = \{0, \ldots, 14\}$, `execution/features.py`)

The action space is the Cartesian product of 5 pace multipliers and 3 order placements, $|\mathcal{A}| = 5 \times 3 = 15$:

$$\texttt{SIZE_MULTS} = (0.0,\ 0.5,\ 1.0,\ 1.5,\ 2.5), \tag{44}$$

$$\texttt{PLACEMENTS} = (\texttt{"market"},\ \texttt{"passive_best"},\ \texttt{"passive_behind"}). \tag{45}$$

Given action index $a \in \{0, \ldots, 14\}$ corresponding to pair (m_a, place_a), let the remaining-over-decisions-left catch-up pace be $b_k = R_k/S_k$ and set the child quantity

$$q_k = \max(0,\ \text{round}(m_a \cdot b_k)). \tag{46}$$

- If $q_k = 0$ (which holds for all three actions with $m_a = 0.0$, indices $0, 1, 2$): return `Decision()` (do nothing; leave any existing resting orders in place to preserve their FIFO queue position!).
- If $q_k > 0$ and place_a = `"market"`: return `Decision(cancel=True, market_qty=`q_k`)`.
- If $q_k > 0$ and place_a = `"passive_best"`: return `Decision(cancel=True, limit_qty=`q_k`, limit_price=None)` (placed at own best: b_t for a buy, a_t for a sell).
- If $q_k > 0$ and place_a = `"passive_behind"`: return `Decision(cancel=True, limit_qty=`q_k`, limit_price=s.behind(1))` (placed 1 tick behind own best: $b_t - 1$ for a buy, $a_t + 1$ for a sell).

12.3 Telescoping Implementation-Shortfall Reward (`execution/env.py`)

A well-known pitfall in RL for optimal execution is reward mismatch: if step rewards include arbitrary heuristic penalties, maximizing cumulative reward no longer minimizes implementation shortfall. In `ExecutionEnv.rollout`, every individual fill $(t_f, p_f, q_f, \text{aggr}_f)$ recorded in `executor.result.fills` between decision k and decision $k+1$ (including any forced market fill at the horizon T, which is credited to the final recorded transition K_{last}) contributes

$$r_k = -\sum_{f \in \text{fills}(k)} \underbrace{\Big(\text{sign} \cdot \frac{p_f - m_{\text{arrival}}}{m_{\text{arrival}}} \times 10^4\Big)}_{\text{fill shortfall in bps}} \cdot \frac{q_f}{Q} \;-\; \mathbb{I}(k = K_{\text{last}}) \cdot \lambda_{\text{inc}} \frac{Q_{\text{incomplete}}}{Q}, \tag{47}$$

where $\lambda_{\text{inc}} = 5.0$ bps.

Theorem (Exact Telescoping Return). *Whenever the order completes in full ($Q_{incomplete} = 0$, which includes any quantity forced by market order at T so that $\sum_k \sum_{f \in fills(k)} q_f = Q$), the undiscounted episode return equals the exact negative implementation shortfall in basis points:*

$$\sum_{k=0}^{K_{\text{last}}} r_k = -\frac{\text{sign}}{m_{\text{arrival}}} \times 10^4 \sum_{k=0}^{K_{\text{last}}} \sum_{f \in \text{fills}(k)} (p_f - m_{\text{arrival}}) \frac{q_f}{Q} = -\,\text{sign} \cdot \frac{\bar{p} - m_{\text{arrival}}}{m_{\text{arrival}}} \times 10^4 = -\text{IS (bps)}. \tag{48}$$

This identity is verified to 10^{-6} in `tests/test_rl.py::test_rollout_reward_equals_negative_shortfall`.

13 Actor–Critic Network and PPO Training Algorithm (`policy.py`, `ppo.py`)

13.1 Network Architecture (`PolicyNet` and `PolicyStrategy`)

`PolicyNet` (`order_book/execution/policy.py`) is a two-hidden-layer tanh multi-layer perceptron with hidden dimension $H = 64$ (6 096 trainable parameters) and separate linear actor and critic heads:

$$\mathbf{h}_1 = \tanh(\mathbf{W}_1 \mathbf{o}_k + \mathbf{b}_1) \in \mathbb{R}^{64}, \tag{49}$$

$$\mathbf{h}_2 = \tanh(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2) \in \mathbb{R}^{64}, \tag{50}$$

$$\text{Action logits:} \quad \boldsymbol{\ell}_k = \mathbf{W}_\pi \mathbf{h}_2 + \mathbf{b}_\pi \in \mathbb{R}^{15}, \tag{51}$$

$$\pi_\theta(a \mid \mathbf{o}_k) = \text{Categorical}(\text{logits} = \boldsymbol{\ell}_k), \tag{52}$$

$$\text{State value:} \quad V_\theta(\mathbf{o}_k) = \mathbf{w}_V^\top \mathbf{h}_2 + b_V \in \mathbb{R}. \tag{53}$$

During training (`deterministic=False`, `recorder` active), `PolicyStrategy.decide` samples $a_k \sim \pi_\theta(\cdot \mid \mathbf{o}_k)$ using a local `torch.Generator` seeded by an integer drawn from the executor's own PCG64 stream (`rng.integers(1 « 31)`), ensuring that stochastic rollouts are 100% deterministic given the simulator seed. During evaluation (`deterministic=True`), it selects the greedy action $a_k = \arg\max_a \ell_{k,a}$.

13.2 GAE and Clipped PPO (`execution/ppo.py`)

Each PPO iteration $i \in \{1, \ldots, I\}$ rolls out $M = 32$ full episodes on seeds drawn from the training pool `TRAIN_SEEDS` $= \{0, \ldots, 999\}$. For each episode trajectory $(\mathbf{o}_0, a_0, r_0, \ldots, \mathbf{o}_{K-1}, a_{K-1}, r_{K-1})$ with terminal bootstrap value $V_K = 0$, `_gae` computes the backward recurrence ($\gamma = 0.999$, $\lambda_{\text{GAE}} = 0.95$):

$$\delta_k = r_k + \gamma V_\theta(\mathbf{o}_{k+1}) - V_\theta(\mathbf{o}_k), \qquad \hat{A}_k = \delta_k + \gamma \lambda_{\text{GAE}} \hat{A}_{k+1}, \qquad \hat{R}_k = \hat{A}_k + V_\theta(\mathbf{o}_k). \tag{54}$$

Across all $N_{\text{batch}} \approx 32 \times 400 = 12\,800$ transitions collected in the iteration, advantages are standardised: $\tilde{A}_k = (\hat{A}_k - \bar{A})/(\text{std}(\hat{A}) + 10^{-8})$. Then `PPOTrainer.update` runs E epochs (`epochs` $\in \{4, 10\}$) of shuffled minibatch updates (minibatch size $B = 512$, Adam optimiser with learning rate lr and global gradient-norm clipping at 0.5) minimising the combined loss:

$$\rho_k(\theta) = \exp\big(\log \pi_\theta(a_k \mid \mathbf{o}_k) - \log \pi_{\theta_{\text{old}}}(a_k \mid \mathbf{o}_k)\big), \tag{55}$$

$$\mathcal{L}_{\text{CLIP}}(\theta) = -\frac{1}{|B|} \sum_{k \in B} \min\Big(\rho_k(\theta)\, \tilde{A}_k,\ \text{clip}(\rho_k(\theta), 1-\epsilon, 1+\epsilon)\, \tilde{A}_k\Big), \qquad (\epsilon = 0.2), \tag{56}$$

$$\mathcal{L}_V(\theta) = \frac{1}{|B|} \sum_{k \in B} \big(V_\theta(\mathbf{o}_k) - \hat{R}_k\big)^2, \qquad \mathcal{H}(\theta) = \frac{1}{|B|} \sum_{k \in B} H\big(\pi_\theta(\cdot \mid \mathbf{o}_k)\big), \tag{57}$$

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CLIP}}(\theta) + c_V \mathcal{L}_V(\theta) - c_H \mathcal{H}(\theta), \qquad (c_V = 0.5,\ c_H = \texttt{entropy_coef}). \tag{58}$$

14 Phase 3 Hyperparameter Sweep and Empirical Results (`outputs/stealth_execution/rl/sweep/`)

Selection bias. The results below were selected and scored on the same seeds, 1000–1039. Re-evaluated by `rl_heldout.py` on 80 seeds never used for selection, these policies are behind TWAP: +0.39 bps [+0.03, +0.76] at $Q/V = 0.2$ and +0.21 bps [−0.00, +0.44] at 0.4. The results are kept as the record of the sweep; do not quote the gains without this.

We ran a grid sweep in the Phase 3 target market (`phase3_config(institutions=1)`: liquid `low_cover` with 1 background institution and 2 detecting market makers at default response) across:

$$\text{lr} \in \{10^{-4},\, 3 \cdot 10^{-4},\, 10^{-3}\} \times c_H \in \{0.003,\, 0.01,\, 0.03\} \times \text{epochs} \in \{4,\, 10\} \times Q/V \in \{0.20,\, 0.40\},$$

with $I = 400$ PPO iterations, $M = 32$ episodes per iteration (12 800 episodes per run), and greedy evaluation every 25 iterations on 40 held-out test seeds (1000..1039). On those exact 40 held-out seeds, the deterministic `twap_market` baseline achieves:

- At $Q/V = 0.20$: $\text{IS}_{\text{TWAP}} = +\mathbf{2.82}$ bps (completion 1.000);
- At $Q/V = 0.40$: $\text{IS}_{\text{TWAP}} = +\mathbf{5.12}$ bps (completion 1.000).

Table 8 reports the top-performing configurations from the completed sweep runs in `outputs/stealth_execution/rl/sweep/summary.json` (ranked by best held-out greedy shortfall among high-completion checkpoints completion ≥ 0.98).

Discussion of the Phase 3 RL results.

1. **Consistent cost reduction across both order sizes:** At $Q/V = 0.20$, the learned policy reduces implementation shortfall from +2.82 bps (TWAP) to +2.40 bps (−0.42 bps, a 14.9% total cost reduction, exceeding the +0.31 bps pure leakage cost by also capturing spread and timing savings via passive orders). At $Q/V = 0.40$, the learned policy reduces IS from +5.12 bps to +4.74 bps (−0.38 bps) at 98.4% natural completion and +4.80 bps (−0.32 bps) at 99.7% natural completion.
2. **Hyperparameter lessons:** Four epochs per rollout batch (`epochs` = 4) is much more stable late in training than `epochs` = 10: with 10 epochs and a high learning rate, the policy can over-optimise a single rollout batch and drift toward lower pre-horizon completion after iteration 200, which is why `train_rl.py` saves `best_policy.pt` at the best held-out evaluation alongside the final `policy.pt`.

Sweep Config Tag	Q/V	Best IS	Compl.	Iter	Last IS	Last Compl.	TWAP	vs. TWAP
`lr1e-3_ent0.003_ep4_sz0.20`	0.20	**+2.40**	0.982	400	+2.40	0.982	+2.82	**−0.42** bps
`lr3e-4_ent0.03_ep10_sz0.20`	0.20	+2.63	0.986	125	+2.77	0.921	+2.82	−0.19 bps
`lr3e-4_ent0.003_ep4_sz0.20`	0.20	+2.64	0.984	325	+2.80	0.982	+2.82	−0.19 bps
`lr1e-3_ent0.01_ep4_sz0.20`	0.20	+2.66	0.988	125	+2.44	0.958	+2.82	−0.16 bps
`lr1e-4_ent0.03_ep10_sz0.20`	0.20	+2.66	0.981	300	+2.69	0.937	+2.82	−0.16 bps
`lr3e-4_ent0.01_ep4_sz0.20`	0.20	+2.67	0.984	100	+2.80	0.918	+2.82	−0.15 bps
`lr1e-3_ent0.003_ep10_sz0.20`	0.20	+2.69	0.990	25	+3.00	0.943	+2.82	−0.13 bps
`lr3e-4_ent0.03_ep10_sz0.40`	0.40	**+4.74**	0.984	125	+5.26	0.954	+5.12	**−0.38** bps
`lr1e-3_ent0.01_ep4_sz0.40`	0.40	**+4.80**	0.997	375	+4.81	0.982	+5.12	**−0.32** bps
`lr3e-4_ent0.03_ep4_sz0.40`	0.40	+4.81	0.987	200	+4.91	0.975	+5.12	−0.31 bps
`lr1e-4_ent0.03_ep10_sz0.40`	0.40	+4.82	0.988	225	+5.09	0.974	+5.12	−0.30 bps
`lr1e-3_ent0.003_ep4_sz0.40`	0.40	+4.83	0.991	250	+5.01	0.970	+5.12	−0.29 bps
`lr1e-4_ent0.01_ep4_sz0.40`	0.40	+4.87	0.994	175	+5.04	0.996	+5.12	−0.25 bps
`lr1e-4_ent0.01_ep10_sz0.40`	0.40	+4.89	0.996	300	+5.06	0.984	+5.12	−0.23 bps
`lr1e-4_ent0.03_ep4_sz0.40`	0.40	+4.89	0.991	350	+5.05	0.990	+5.12	−0.23 bps

Table 8: Hyperparameter sweep results on 40 held-out test seeds (1000..1039) in the Phase 3 `low_cover` market (`outputs/stealth_execution/rl/sweep/summary.json`). All shortfalls include forced market completion at the horizon.

Part V
The Counterfactual Machinery: Reaction Channels, the Ghost Arm, Paired Seeds

The leakage counterfactual of §11.1 answers one question: what does it cost that detectors can see the executor? It cannot answer the larger one, because `Market.hide` only blinds the detector. A hidden executor still moves the mid, so trend followers react to it and market makers still skew on the inventory it pushes onto them, and all of that sits inside what E3 calls impact cost. This part builds the instrument for the larger question: how much of implementation shortfall is liquidity the order mechanically consumed, and how much is the market's reaction, split into inventory, price, detection and predation.

The code is in `order_book/counterfactual.py` (the worlds), `ghost.py` (the copy book) and `rng.py` (the draws). This part documents that machinery, which any study can use. The first study built on it, with its accounting, experiments, results and caveats, is in `papers/decomposition/` (working notes: `paper/notes.pdf`).

15 Reaction Channels and Arms

Every run steps two worlds in lockstep: a **reference** world, Arm 0, that never contains an executor, and a **treated** world where the executor trades. Four switches (`Channels`) decide which reactions to the executor exist in the treated world:

Channel	On	Off
I inventory	makers skew their reservation on their holding in the treated world	makers skew on their holding in Arm 0 (`MarketMaker.skew_account`)
P price	the background lives on the true book and reads its public state	the background acts in Arm 0; its order flow is replayed on a copy of the book
D detection	the makers' flow detector reads the complete flow	it reads the flow without the executor (`hide`)
A predation	the anticipator trades, on a detector that reads the complete flow	the anticipator stays dormant

All four off is the **ghost arm**; all four on is the full A4 market. The switches are independent, which makes $2^4 = 16$ worlds per seed. The population always contains the anticipator, dormant when off, so every world shares one scheduler permutation; removing it would change the length of the permuted list and with it every agent's position.

The natural ladder of the research plan is the chain of nested arms in Table 9.

Table 9: The arms. Arm k adds one thing to Arm $k-1$.

Arm	`Channels.key`	Treated world
0		no executor
1	`--`	ghost: the executor fills against a live copy of Arm 0's book; the background observes and acts on Arm 0
2	`-P-`	true book, reservation on Arm-0 inventory, detector blind
3	`IP-`	true book and inventory, detector blind (what E3 calls "invisible")
4	`IPD-`	the detector sees the executor (A1)
5	`IPDA`	the anticipator trades (A4)

What the increments mean. The increments are named here by what actually changes between two arms. The research plan's table attaches "inventory skew" to the step into Arm 2 and "chartist response" to the step into Arm 3, but Arm 2 as written freezes inventory, so Arm 2−Arm 1 is the move onto the true book and Arm 3−Arm 2 is the inventory channel. The order the plan's labels name, inventory first, is also available (`PLAN_LABEL_ORDER`, reported as `ladder_plan_order`), since both chains are paths through the same 16 cells.

The price channel is not only chartists. It holds every reaction to the true book: chartists chase the move, fundamentalists lean against it, makers re-centre their quotes on it and thin out with its volatility, noise traders place relative to the moved quotes. Three controls outside the factorial look inside it. Each repeats Arm 2 with one group pointed at Arm 0's public state while everything else stays live: `-P-c` blinds chartists' momentum, `-P-m` blinds the makers' mid and volatility, `-P-f` blinds the mid fundamentalists compare with their belief. The control `IPD-b` is Arm 4 with an anticipator that trades but cannot see the executor, which separates the effect of having it trade on natural flow from the effect of it seeing the executor.

16 The Ghost Arm

`GhostMarket` forks a `Market` after the burn-in: a deep copy of the book, the accounts and the public state. It then acts as the source market's `mirror`, which is told of every order and cancel that reached the source book. The background keeps acting on the source, which never learns the executor exists; the executor trades only on the copy. The two books diverge as soon as it trades, so replaying an order needs a rule. The rule treats each lot as what it was in Arm 0:

R1, takers. A lot that traded in Arm 0 takes liquidity. It is replayed immediate-or-cancel at its own limit (market orders at no limit) and walks the copy's book by the ordinary price–time matching. If the copy has nothing at that price it lapses. Its sender wanted to trade now; resting it would create a quote that no owner will ever cancel.

R2, makers. A lot that rested in Arm 0 makes liquidity. It rests at its own price, clamped post-only to one tick behind the copy's opposite best if it would cross there (the rule makers already apply to themselves), and is dropped if that price is outside the copy's window.

R3, removals. A cancel, eviction or self-trade removal in Arm 0 removes the order's counterpart in the copy, if it is still there.

R4, orphans. An order that fully traded in Arm 0 but still rests in the copy has no owner who will cancel it. This arises when the executor's passive order took the fill the Arm-0 order got. It stays, because the executor really did add that much liquidity, but expires after `orphan_ttl` steps (default 50, the order of magnitude of a background order's life) so it cannot pile up into a wall no real population would maintain.

R5, budgets. Only the executor is budget-checked. The background's accounts in the copy are shadows of orders funded in Arm 0 and may go negative; cash and shares are still conserved across the copy.

With no executor the copy reproduces the source exactly; `tests/test_counterfactual.py` asserts this tick for tick. `GhostMarket.stats` counts every intervention of the rule (lapsed lots, clamps, drops, orphans, expiries), and each run reports them.

17 Realising the Channels

Each pair of agents acts in the same scheduled position, treated first, so a treated agent reads the reference exactly as its twin is about to. This is why the worlds coincide in every configuration when there is nothing to react to.

With P on, the treated world is a clone of the reference taken from the burn-in snapshot, with its agents acting on what they see there. I off points each maker's skew at its twin's account. Only the reservation is frozen: the `max_inventory` pull is a risk limit and keeps using the maker's real holding. D off leaves the executor out of the flow the makers' detector reads.

With P off, the background's orders come from Arm 0, so I and D cannot change a decision. They shade each replayed maker quote instead, by the difference between the copy's terms and Arm 0's, using the maker's own formula (`MarketMaker.shade`). With inventory q and signal s in the copy, and q_0, s_0 in Arm 0,

$$\Delta_{\text{res}} = \text{clamp}(q_0\,\rho) - \text{clamp}(q\,\rho) + \kappa_s(s - s_0), \qquad \rho = \gamma\,\max(\sigma^2_{\text{fast}}, \sigma^2_{\text{min}})\,h,$$
$$p' = \text{round}\big(p + \Delta_{\text{res}} \pm (w(s) - w(s_0))\big), \qquad x' = \max\big(1, \text{round}(x\,c(s)/c(s_0))\big),$$

where the sign is + on the ask, and w and c are the widening and size multiplier the signal puts on that side (zero and one on the unpressured side). No order is created or retimed, and the shade is exactly zero when nothing has diverged, so a channel switched on with no divergence changes nothing. A shaded quote waits for the maker's next requote in Arm 0 to take effect.

Predation runs the same way in both: a live anticipator in the treated world, on its own detector. Makers and anticipator therefore switch independently, and $\{A\}$ without $\{D\}$ is well defined.

The executor reads the treated public state, with `imbalance_all` in place of `imbalance`. Without this, hiding it would also change what it decides on, since a strategy that reads the flow

would stop seeing its own part of it. That was a defect in E3 as first run: `footprint` showed nonzero leakage under A0, up to +0.24 bps, where nobody reads a detector and leakage must be exactly zero. E3 was regenerated after the fix: non-adaptive strategies are unchanged to the last digit, and A0 leakage is now exactly zero for every strategy and size.

18 A Paired-Seed Protocol for Agent-Based Counterfactuals

This section is written to stand alone, as the protocol section of a paper.

An agent-based market is a stochastic program. To estimate the effect of an intervention, one runs it with and without the intervention on shared random numbers, because the effect of interest is far smaller than the program's noise. Shared seeds are only a variance-reduction device if the two runs actually share noise. With stateful generators they stop doing so once the intervention changes control flow: a market maker that requotes one extra time consumes one extra draw, and every later draw of that agent shifts. The two runs then differ for reasons that have nothing to do with the intervention, which Buffalo, Pearson and Klein (2026) call causal incoherence. One stream per agent, the usual remedy, only confines the damage to one agent per event.

Construction. Each draw is made a pure function of its address. A `KeyedStream` holds a Philox key $(\text{seed}, \text{crc32}(\text{label}))$ and no sequential state between calls. Asking for a draw names its step, its purpose (a small integer per decision an agent draws for) and, for a purpose drawn more than once per step, an index; the generator is rekeyed to the counter $(0, \text{step}+1, \text{purpose}, \text{index})$, and any partial word left over from an earlier address is discarded. Philox increments the first counter word per output block, so a sequence inside an address cannot run into another address. Rekeying costs about 1 μs, and a whole simulation runs about 18% slower than with stateful streams.

Properties , each asserted in `tests/test_rng.py`:

1. What an agent draws at an address does not depend on how many draws it made before, or on what anyone else did.
2. Identity is by label, not by position: adding or removing agents leaves everyone else's draws alone, and the executor's stream is the same whether or not the population contains an anticipator.
3. Arrays drawn at one address are stable in their prefix, with one address per attribute of an order. A Poisson count that differs between two runs changes only the tail of an agent's orders.
4. Poisson counts at one address are monotone in the rate (NumPy's multiplication sampler at rates below 10), so a small difference in activity gives a small difference in orders. This is the monotone coupling common random numbers want.
5. One code path serves both modes: `keyed(rng, step, purpose)` returns a stateful generator unchanged. Stateful runs are byte-identical to before this machinery existed, which pins every earlier result.

What it does not do. An agent whose decision really differs still sends different orders, and a market that has diverged keeps diverging. Keying removes the spurious part of divergence, not the causal part. Hence the next two measurements.

Variance reduction. For a contrast between arms a and b over seeds,

$$\text{VR} = \frac{\text{Var}(a) + \text{Var}(b)}{\text{Var}(a-b)},$$

the factor by which pairing shrinks the variance of the estimated difference against independent seeds, as Klein et al. measure it. `crn_diagnostics` reports it with a seed-bootstrap CI. It exceeds

one exactly when $\mathrm{Cov}(a, b) > 0$, which is Sharma's positive-correlation condition, and we report the correlation with its CI and require the whole interval above zero. Both are computed per contrast and pairing scheme.

Coupling. `flow_coupling` measures divergence directly: the Jaccard overlap of the multiset of background orders $(\text{step}, \text{client}, \text{side}, \text{size})$ between the two worlds, per fifth of the window. This shows where a shifted stream loses the pairing for good, which an aggregate variance ratio cannot.

Part VI
Verification, Test Suite, and Complete Reproduction Guide

19 Automated Test Suite (`tests/`)

Running `python -m pytest` executes 323 unit and integration tests in about 45 seconds, across `tests/` and `papers/*/tests/`:

`test_book.py` OrderBook rules: empty and one-sided states, exact window boundaries and reported evictions, FIFO and partial fills on both sides, price protection of crossing limits, all four self-trade prevention modes after an external partial fill, cancel/modify priority preservation, atomic rejection of invalid modifications, input validation including NumPy integers, snapshot/array isolation, overflow of aggregate volume, and a 600-action random fuzz sequence asserting full book invariants after every single action.

`test_market.py` Exact cash and share reservation bookkeeping under every removal path (fill, cancel, STP, window eviction, budget cap) and ring-buffer flow window updates.

`test_simulator.py` Determinism, cash/share conservation, snapshot/restore and in-place `reseed`, analysis-mode vs. training-mode logging, population construction with shared detector registry, frozen and replaceable configuration, flow detector thresholding and saturation.

`test_foundations.py` Byte-for-byte event-log replay from `(config, seed)`, from a restored snapshot, and after `reseed`; stream independence from population changes; invariants asserted after every step; and the **no-hidden-information guard** (which proxies `Market`, `OrderBook`, `FundamentalValue`, and `MetaorderLog` to raise an error if any agent, detector, or `ExecutorAgent` ever accesses private identities, other accounts, the tape, or ground-truth labels).

`test_agents.py` All five institutional execution styles complete on both sides, are labelled, and leave no resting order; log-normal sizes, balanced sides, inventory reversion; order anticipator directional and contrarian-first behaviour; market-maker volatility pull cooldown and fragility scaling; private valuations; external agent stream non-interference.

`test_presets.py` Composition of `regime()` and `adversary()` preserving thin-book market-maker sizes and custom detecting-maker counts.

`test_execution.py` All 12 baseline strategies complete Q on both sides, leave no resting orders, and report exact tape-consistent IS; hard limits (resting orders, through-ticks, participation) bind; `Market.hide` removes the executor from flow statistics and detectors while keeping price and volume identical under A0; Almgren–Chriss and Obizhaeva–Wang schedule identities; resilience exponential fit; `FillProber` and calibration serialization.

`test_detectors.py` `FeatureTracker` rolling features and regularity; A2 detectors (`runs`, `cusum` alarm/restart/decay, `volume_clock`, `regularity`); A3 logistic classifier training to AUC $>$ 0.9 on synthetic separable data and weight serialization.

`test_scoring.py` Mann–Whitney ROC AUC (perfect, inverted, tied, degenerate), detection delay, false-alarm rate, activity labels, percentile bootstrap CI, paired difference, CVaR_{95},

and Holm step-down adjustment.

test_experiments.py (papers/stealth_execution/tests/) Episode parity, deterministic JSON-serializable `EpisodeResult`, exact equality of visible and invisible runs under A0, E2/E3/E7/E7b runners and summaries, impact paths.

test_env.py Legacy Gymnasium wrapper `OrderBookEnv` seeding, bounds, and pickling.

test_rl.py Phase 3 RL suite: 13-feature vector shape, finiteness, and buy/sell sign symmetry; 15-action decision mapping; exact telescoping equality $\sum_k r_k = -\text{IS}$ (bps); seed-deterministic rollouts; `PolicyNet` save/load round-trip; `make_strategy("rl@...")`; PPO weight update and finite losses; `phase3_config`; and `sweep_rl` job tagging and ranking.

test_rng.py Draws depend only on their address, with no leak of buffered words; prefix stability; Poisson counts monotone in the rate; one extra requote shifts every later draw of a stateful stream and none of a keyed one; determinism, snapshot and reseed in counter mode; streams keyed by label.

test_simulator_version.py A fingerprint per `SIMULATOR_VERSION` of seeded runs in both RNG modes, an execution episode and paired runs. A change in behaviour fails until the version is bumped and the new fingerprint recorded.

test_layout.py The library never imports a paper, and papers never import each other.

test_runner.py Job order, the result log surviving a job killed mid-write, manifests, refusal to resume across simulator versions, and the commit recorded for a cluster without `.git`.

test_ghost.py One test per matching rule R1–R5, on hand-built books: lapsing at the limit, walking further, the split of an order's traded and rested parts, the post-only clamp, the window drop, cancels and evictions of counterparts, orphans with and without expiry, shadow budgets and conservation; the book-level `ioc`; and full book, reservation and conservation invariants in the copy with a trading executor.

test_counterfactual.py With no executor every configuration equals Arm 0 tick for tick in both RNG modes; Arm 0 inside a paired run equals the same seed with no executor at all; Arm 4 equals a plain single-world run exactly; exact nulls (detection with no detecting maker, inventory with no inventory pricing, predation with no trading, and an instant fill costing the same in all 16 worlds); hiding changes no decision of the executor; each switch does what it says; every baseline and the learned policy complete in the ghost and the full market.

test_decomposition.py (papers/decomposition/tests/) Shapley values on known games and against the average over all orderings; the accounting identity on results with known values and on the simulator; ladder order dependence; controls; exponents and reversion recovered per arm; executor routes; two-sided p-values, the joint ratio bootstrap and the variance-reduction diagnostics; the protocol and replication summaries; parallel and serial runs identical.

test_detection_curve.py (papers/decomposition/tests/) The dialled TWAP's endpoints are exactly `twap_market` and `twap_passive`; timing jitter keeps the schedule on pace; firing, latency, duty and signal are scored by direction; the curve end to end with the rule-based and learned detectors.

20 Complete Command Reference for Reproducing Every Phase

```
# --- Environment & Test Suite ---
uv pip install -e .                                # install package + numpy, matplotlib,
    gymnasium, torch, wandb
.venv/bin/python -m pytest                         # 323 tests in ~45s

# --- Phase 1: Simulator Validation & Square-Root Impact ---
.venv/bin/python -m scripts.benchmark              # speed benchmark (~15,000 steps/s) +
    cProfile
.venv/bin/python -m scripts.validate --seeds 5 --steps 30000
```

```
.venv/bin/python -m scripts.impact --seeds 20 --styles twap_market twap_passive pov
.venv/bin/python -m scripts.run                    # single 10k-step diagnostic plot -> outputs/
    simulation.png
.venv/bin/python -m scripts.animate                # live order-book video -> outputs/
    simulation.mp4

# --- Phase 2: Calibration, Detector Training, Detectability & Leakage ---
.venv/bin/python -m scripts.calibrate              # fit A0 parameters -> outputs/calibration.
    json
.venv/bin/python -m scripts.train_detector         # train A3 classifier -> outputs/detectors/
    learned.npz
.venv/bin/python -m papers.stealth_execution.scripts.e2_detectability --workers 8
.venv/bin/python -m papers.stealth_execution.scripts.e3_leakage --workers 8 --paths
.venv/bin/python -m papers.stealth_execution.scripts.e7_regimes --workers 8
.venv/bin/python -m papers.stealth_execution.scripts.e7_strength --workers 8

# --- Phase 3: RL Executor Training, Hyperparameter Sweep & Counterfactual Eval ---
caffeinate -i .venv/bin/python -u -m papers.stealth_execution.scripts.train_rl \
  --iterations 400 --episodes 32 --epochs 4 --lr 1e-3 --entropy 0.003 --size 0.20 \
  --eval-every 25 --eval-seeds 40 --wandb --out rl/best_sz020

caffeinate -i .venv/bin/python -u -m papers.stealth_execution.scripts.sweep_rl \
  --workers 6 --wandb --sync-existing --wandb-project stealth-execution

# Score a trained RL checkpoint against baselines via the E7b leakage counterfactual:
.venv/bin/python -m papers.stealth_execution.scripts.e7_strength --workers 8 \
  --strategies rl@outputs/stealth_execution/rl/sweep/lr1e-3_ent0.003_ep4_sz0.20/policy.pt
    twap_market twap_passive

# --- The decomposition study: see papers/decomposition/README.md; the full run goes through
    hpc/README.md ---
.venv/bin/python -m papers.decomposition.scripts.decompose --quick
.venv/bin/python -m papers.decomposition.scripts.make_tables
cd papers/decomposition/paper && latexmk -pdf notes.tex

# Rebuild this PDF manual:
cd docs && latexmk -pdf simulator.tex
```